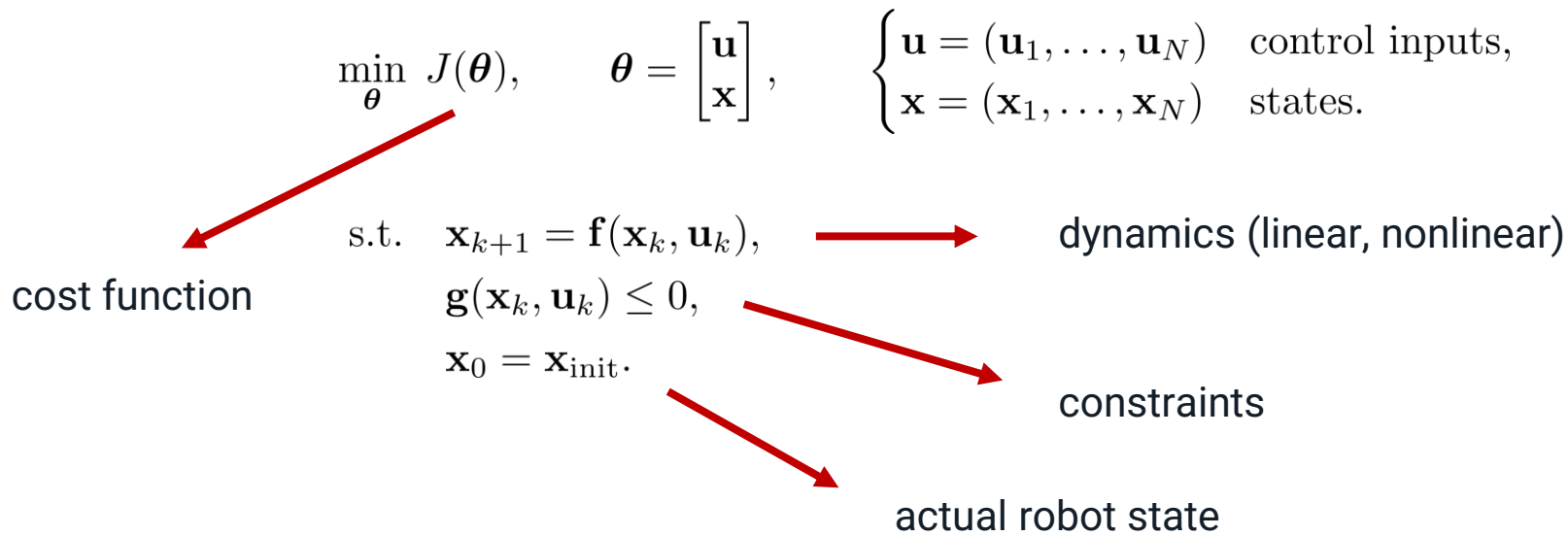


Introduction to Sampling-based Control

Giulio Turrisi, Postdoctoral Researcher
Italian Institute of Technology

- General formulation of optimal control



- The cost function is (typically) a finite sum of terms for N steps

$$J(\boldsymbol{\theta}) = \sum_{k=0}^N \ell(\mathbf{x}_k, \mathbf{u}_k)$$

Three red arrows point from the cost function to the following terms:

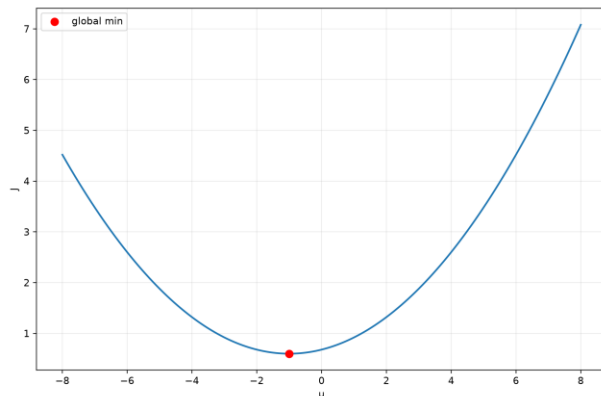
- $\ell(\mathbf{x}_k, \mathbf{u}_k) = \|\mathbf{x}_k - \mathbf{x}_k^{\text{ref}}\|_Q^2$ tracking
- $\ell(\mathbf{x}_k, \mathbf{u}_k) = \|\mathbf{u}_k\|_R^2$ minimize input
- $\ell(\mathbf{x}_k, \mathbf{u}_k) = \frac{1}{d(\mathbf{x}_k, p)}$ collision avoidance

- Depending on the cost, dynamics, decision variable, your **OCP can have different characteristics**

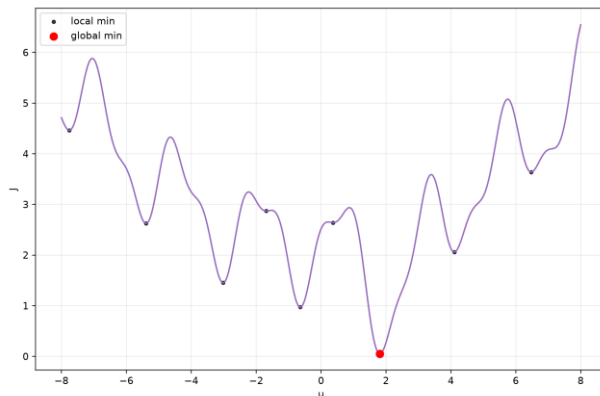
Optimal Control Recap

- For simplicity, let's focus on few of them:

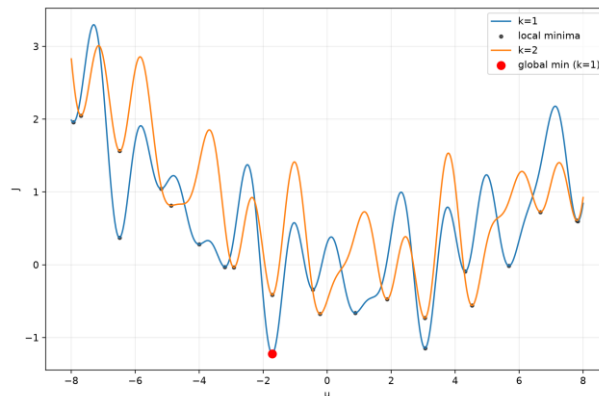
convex quadratic: easy,
one minimum



non-convex: more
difficult, multiple minima

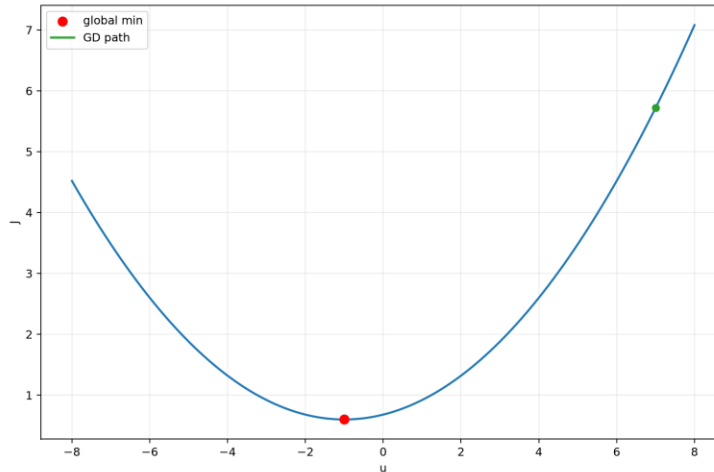


mixed-integers: more
difficult, multiple minima



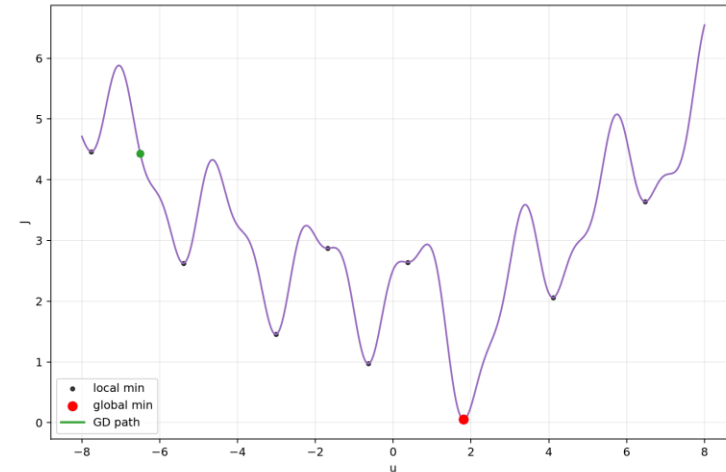
Type of Opt. Problem

convex quadratic: easy, one
minimum



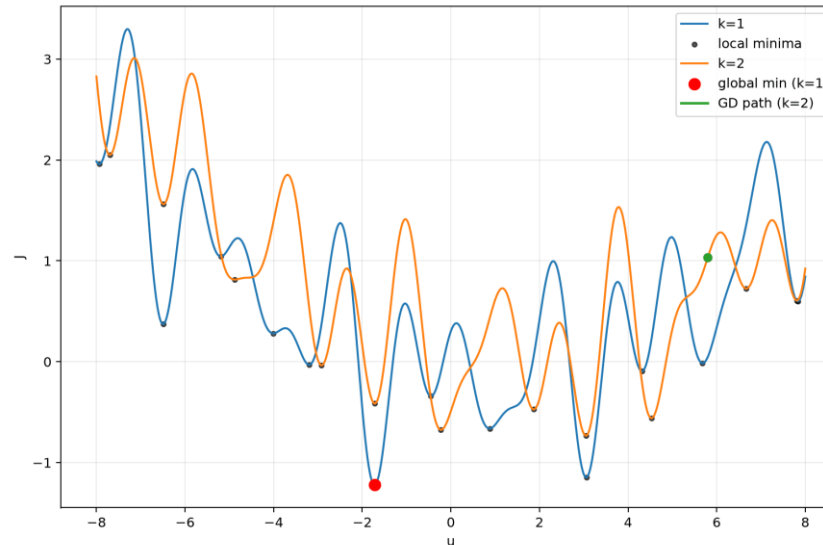
without constraints, you can
find it in one step!

non-convex: more difficult,
multiple minima



Gradient Descent, starting
from two different points!

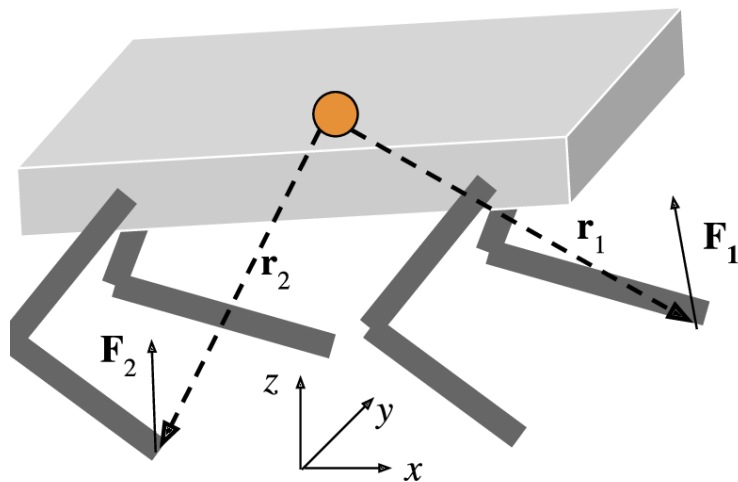
mixed-integers: more difficult,
multiple minima



SQP has problem with it! Usually solved by **simplifying** the problem

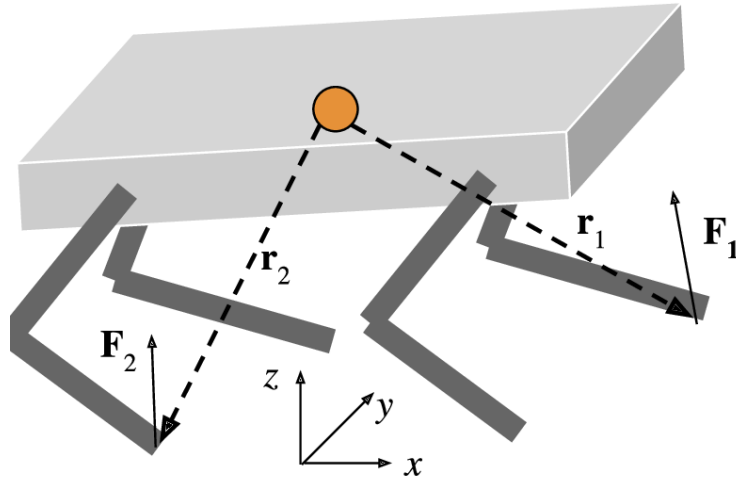
How much common are these cases?

Centroidal model:



- only the robot's torso is considered
- inertial effects of leg movement are ignored
- can optimize foothold's location $\mathbf{r}$ together with Ground Reaction Forces $\mathbf{F}$

How much common are these cases?



SRBD model:

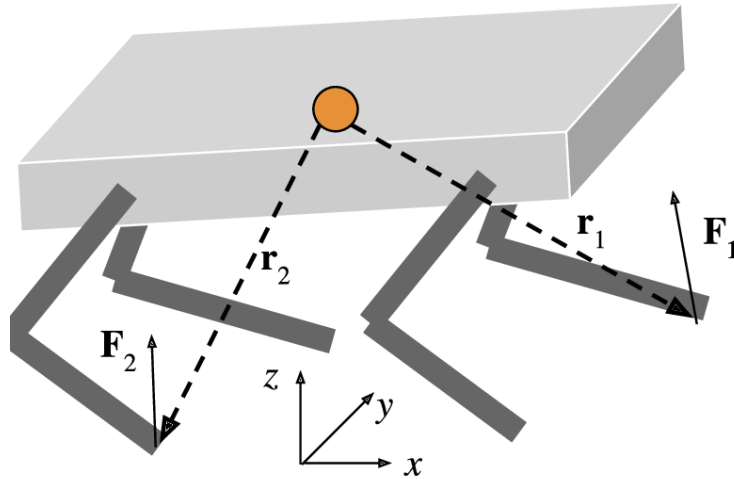
$$\mathbf{u} = [\mathbf{r}, \mathbf{F}, \delta] \quad \longrightarrow \text{inputs}$$

$$m(\ddot{x}_{com} + g) = \sum_{i=1}^c F_i \delta_i \quad \longrightarrow \text{linear part}$$

$$I_G \dot{\omega}_b \simeq \sum_{i=1}^c (F_i \times r_i) \quad \longrightarrow \text{angular part}$$

How much common are these cases?

SRBD model:

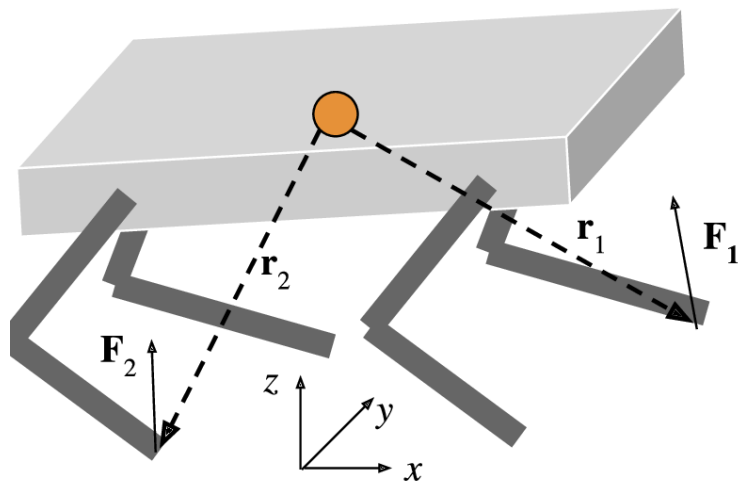


$$m(\ddot{x}_{com} + g) = \sum_{i=1}^c F_i \delta_i \longrightarrow \text{linear part}$$

Delta is discrete $[0,1]$
(**mixed-integer program!**)

How much common are these cases?

SRBD model:



$$I_G \dot{\omega}_b \simeq \sum_{i=1}^c (F_i \times r_i) \quad \longrightarrow \quad \text{angular part}$$

Bilinear product between r and F
(**non-convex!**)

Some questions arise:

1. at runtime, do we need to find all the minimum?

- usually in MPC better to be fast than super optimum

2. is any minimum acceptable?

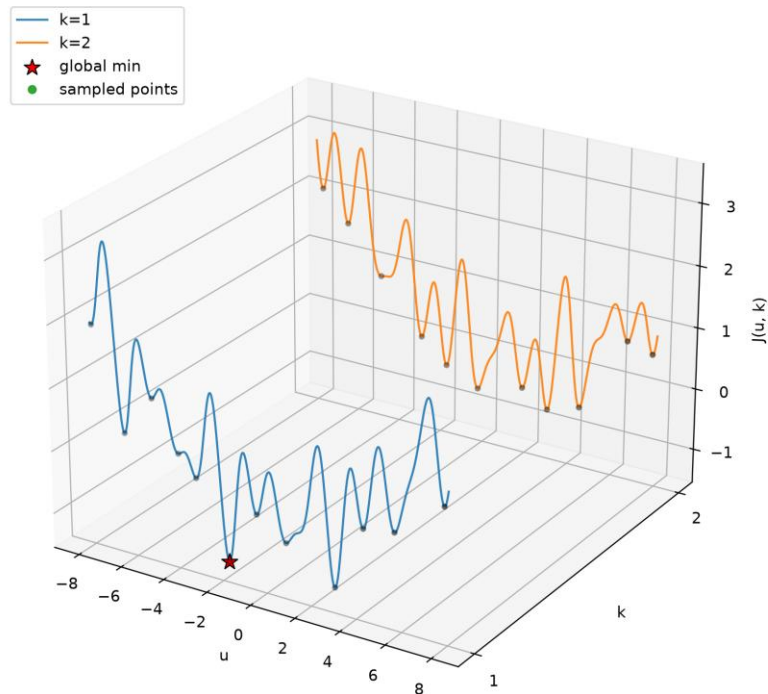
- not always!

3. how do we jump from one minimum to the other?

1. fixing discrete parameters (like in pre-planned gait)
2. smart warm start (SQP it will typically converge to a local solution)
3. exploration (try multiple solution at once. Is it possible?)

Simple idea on how to “solve” this:

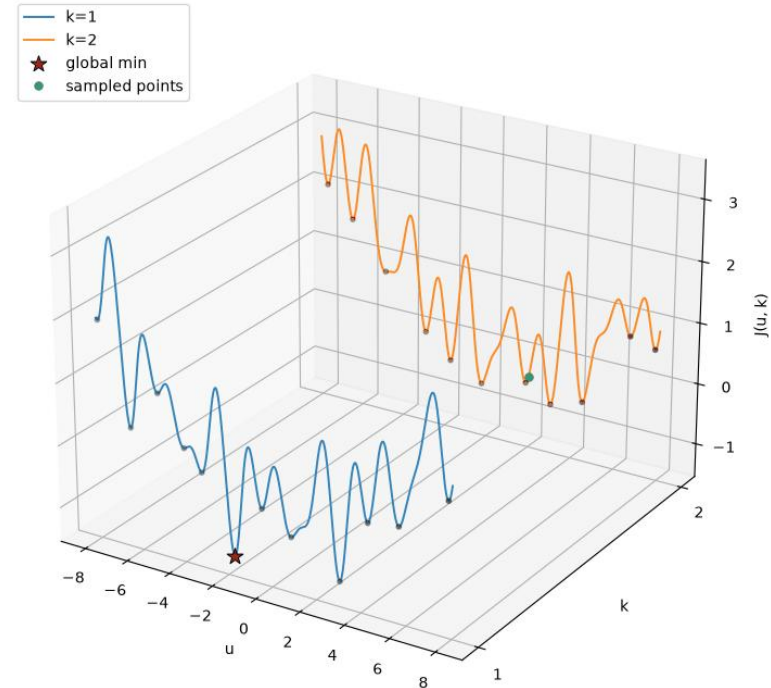
What if we have enough power to
sample a lot of different solutions...

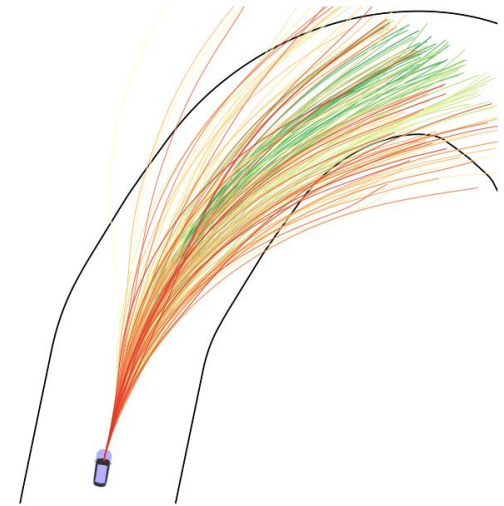
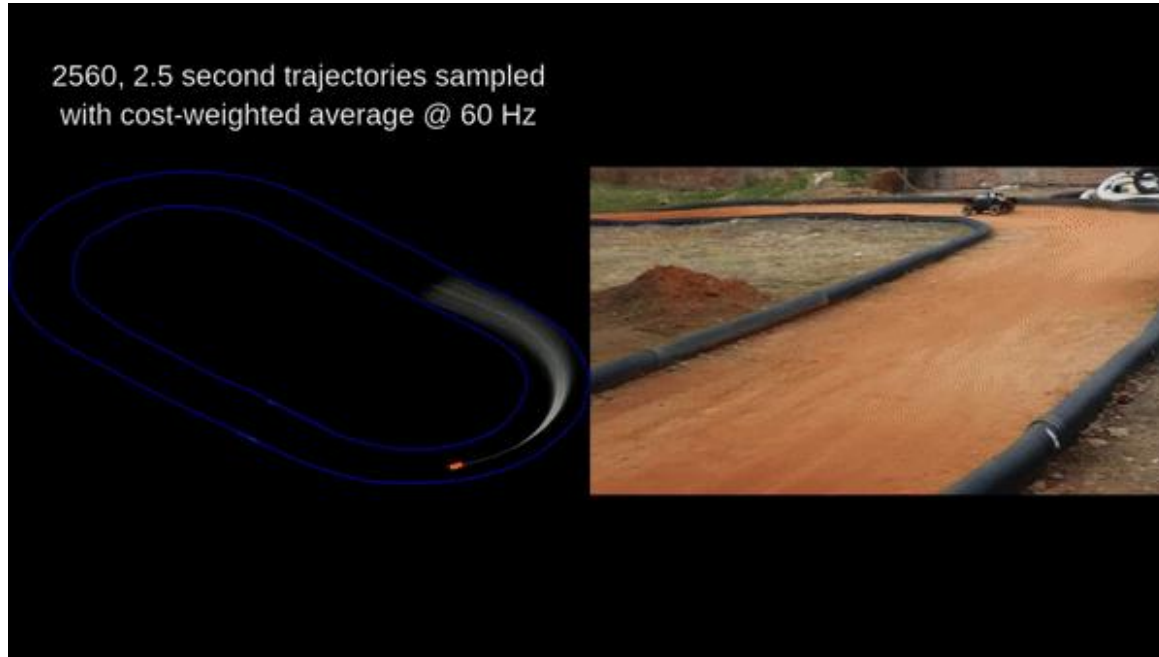


Simple idea on how to “solve” this:

What if we have enough power to
sample a lot of different solutions...

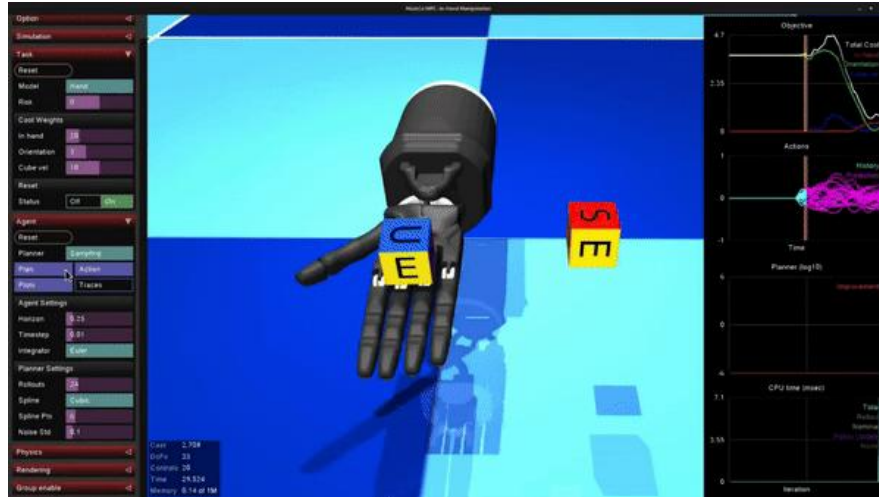
...in **parallel**?



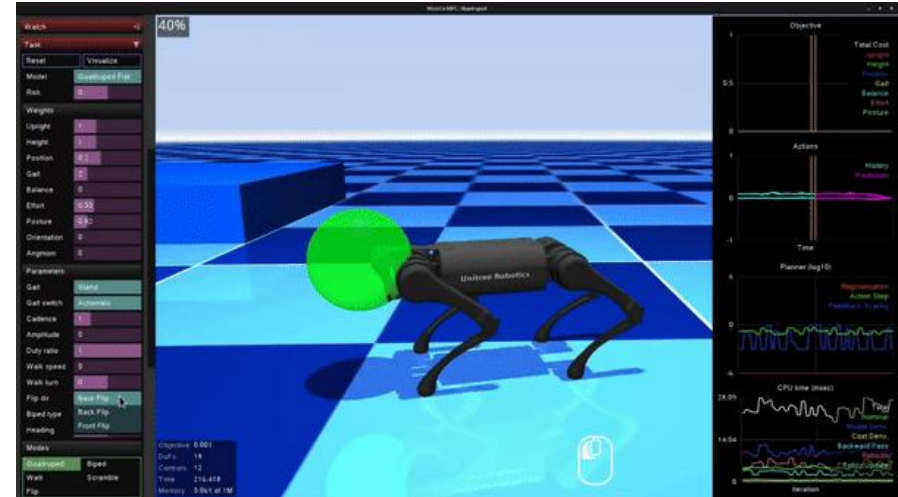


G. Williams et al., “Aggressive driving with model predictive path integral control”, 2016 IEEE International Conference on Robotics and Automation (ICRA), 2016

Applications



Rubik's Cube
manipulation



Quadruped whole-body motions



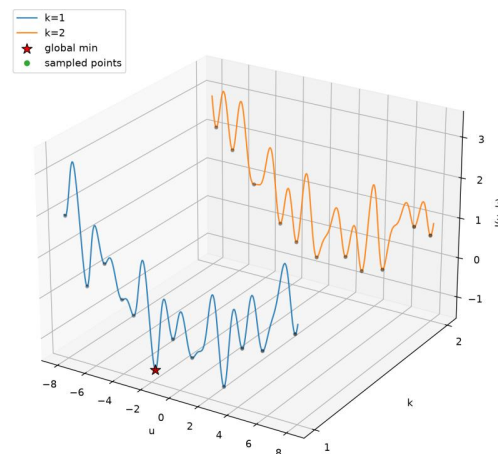
- Random sampling control
- Practical limitations
- CEM
- MPPI
- State-of-the-Art sampling in Legged Robots

- **Random sampling control**
 - Sampling instead of gradients
- Practical limitations
- CEM
- MPPI
- State-of-the-Art sampling in Legged Robots

Simple idea on how to “solve” this:

1. Sample a lot of different solutions

$$\theta^i = (u^i, k^i),$$

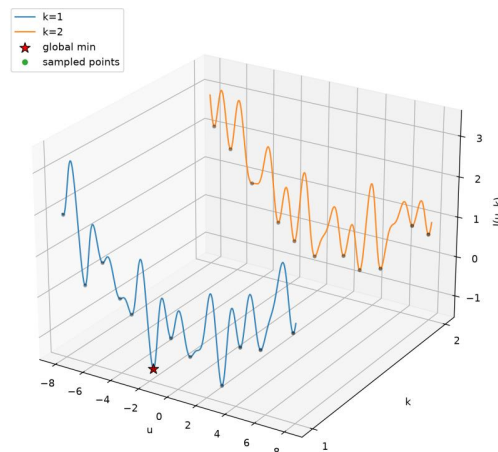


$$\begin{aligned}\theta^1 &= (3, 0), \\ \theta^2 &= (3.2, 1), \\ \theta^3 &= (1.4, 1), \\ \theta^4 &= (-2.2, 0), \\ &\vdots \\ \theta^K &= (0.1, 1)\end{aligned}$$

Simple idea on how to “solve” this:

1. Sample a lot of different solutions

2. Evaluate them



$$\theta^1 = (3, 0),$$

$$\theta^2 = (3.2, 1),$$

$$\theta^3 = (1.4, 1),$$

$$\theta^4 = (-2.2, 0),$$

$$\vdots$$

$$\theta^K = (0.1, 1)$$

$$\mathbf{J}^1 = 0,$$

$$\mathbf{J}^2 = 0.3,$$

$$\mathbf{J}^3 = 0.5,$$

$$\mathbf{J}^4 = -1,$$

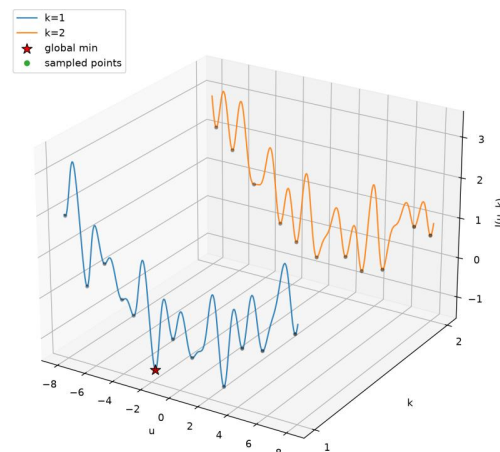
$$\vdots$$

$$\mathbf{J}^K = -0.7$$

Simple idea on how to “solve” this:

1. Sample a lot of different solutions
2. Evaluate them

3. Then pick the best



$$\theta^1 = (3, 0), \quad \mathbf{J}^1 = 0,$$

$$\theta^2 = (3.2, 1), \quad \mathbf{J}^2 = 0.3,$$

$$\theta^3 = (1.4, 1), \quad \mathbf{J}^3 = 0.5,$$

$$\theta^4 = (-2.2, 0), \quad \mathbf{J}^4 = -1,$$

$$\mathbf{J}^4$$

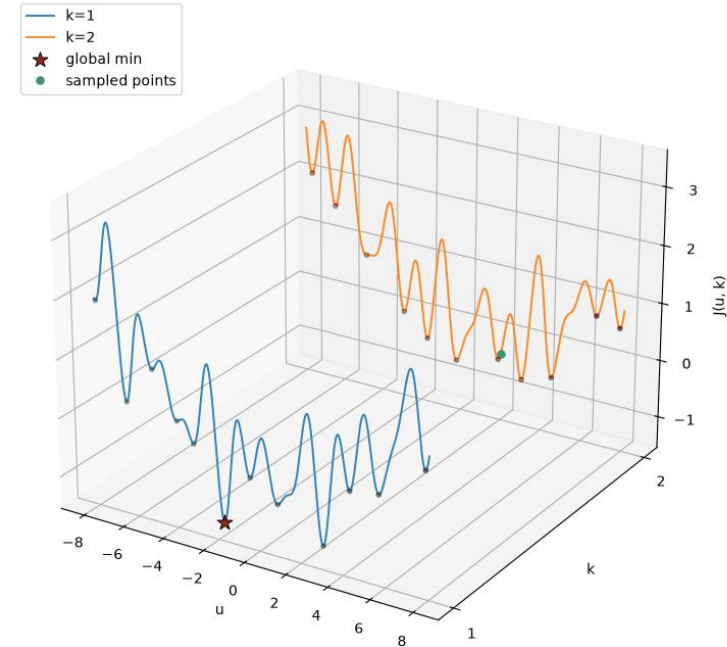
$$\vdots$$
$$\vdots$$

$$\theta^K = (0.1, 1)$$

$$\mathbf{J}^K = -0.7$$

Simple idea on how to “solve” this:

1. Sample a lot of different solutions
2. Evaluate them
3. Then pick the best



Algorithm 1 Generic iteration of Random Sampling

At each control loop:

- | | |
|---|--------------|
| 1: $\theta^{k=1,\dots,K} \sim p(\theta)$ | ▷ Sampling |
| 2: $J_k = \text{CostEval}(\theta^k)$ | ▷ Evaluation |
| 3: $\theta^* \leftarrow \text{select}(\theta^k, J^k)$ | ▷ Selection |
| 4: $\text{SendToMotors}(\theta^*)$ | ▷ Control |
-

We usually are in an MPC scenario, so we need to look ahead **N steps**!



$\theta^i = [u_0, u_1, \dots, u_N]$
If u has dimension M ,
we **sample $K \times N \times M$ numbers**!

..or for simplicity, we say “**K trajectories**”

Algorithm 1 Generic iteration of Random Sampling

At each control loop:

- 1: $\theta^{k=1,\dots,K} \sim p(\theta)$ ▷ **Sampling**
 - 2: $J_k = \text{CostEval}(\theta^k)$ ▷ **Evaluation**
 - 3: $\theta^* \leftarrow \text{select}(\theta^k, J^k)$ ▷ **Selection**
 - 4: $\text{SendToMotors}(\theta^*)$ ▷ **Control**
-

generation of random trajectories
from a distributions (uniform,
gaussian..)

$$\theta^1, \theta^2, \theta^3, \dots$$

Algorithm 1 Generic iteration of Random Sampling

At each control loop:

1: $\theta^{k=1,\dots,K} \sim p(\theta)$

2: $J_k = \text{CostEval}(\theta^k)$

3: $\theta^* \leftarrow \text{select}(\theta^k, J^k)$

4: $\text{SendToMotors}(\theta^*)$

▷ Sampling

▷ Evaluation

▷ Selection

▷ Control

evaluation of each samples over the
robot dynamics f and the **cost
function**

$$x^1 = f(x^0, \theta^1)$$

$$x^2 = f(x^0, \theta^2)$$

$$x^3 = f(x^0, \theta^3)$$

⋮

Algorithm 1 Generic iteration of Random Sampling

At each control loop:

1: $\theta^{k=1,\dots,K} \sim p(\theta)$

▷ **Sampling**

2: $J_k = \text{CostEval}(\theta^k)$

▷ **Evaluation**

3: $\theta^* \leftarrow \text{select}(\theta^k, J^k)$

▷ **Selection**

4: $\text{SendToMotors}(\theta^*)$

▷ **Control**

take the best θ^*

Algorithm 1 Generic iteration of Random Sampling

At each control loop:

- | | |
|---|---------------------|
| 1: $\theta^{k=1,\dots,K} \sim p(\theta)$ | ▷ Sampling |
| 2: $J_k = \text{CostEval}(\theta^k)$ | ▷ Evaluation |
| 3: $\theta^* \leftarrow \text{select}(\theta^k, J^k)$ | ▷ Selection |
| 4: $\text{SendToMotors}(\theta^*)$ | ▷ Control |



apply on the real system

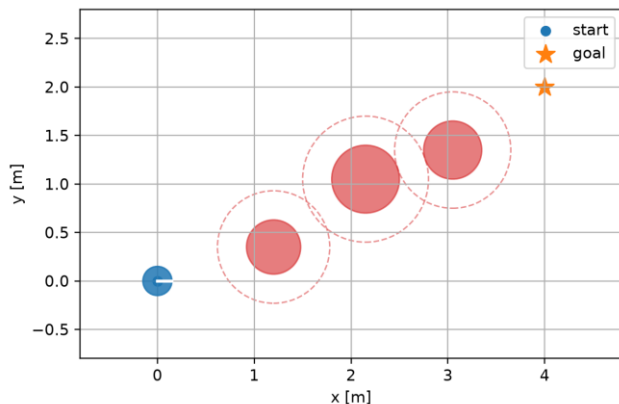
$\theta = [v, w]$ $\longrightarrow$ 2 control input

$$x_{k+1} = x_k + v_k \cos(\psi_k) dt$$

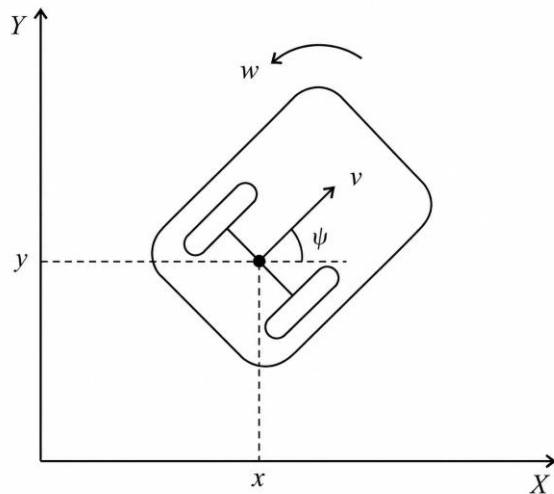
$$y_{k+1} = y_k + v_k \sin(\psi_k) dt \longrightarrow$$

$$\psi_{k+1} = \psi_k + w_k dt$$

kinematic model
+ euler



Task: reaching the goal while avoiding
static obstacles



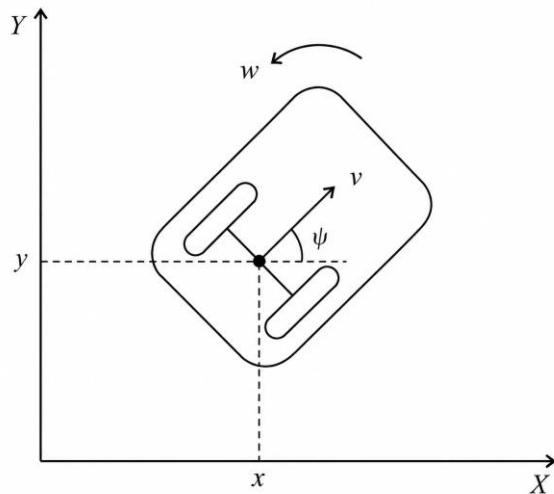
$\theta = [v, w]$ $\longrightarrow$ 2 control input

$x_{k+1} = x_k + v_k \cos(\psi_k) dt$
 $y_{k+1} = y_k + v_k \sin(\psi_k) dt$
 $\psi_{k+1} = \psi_k + w_k dt$ $\longrightarrow$ kinematic model
+ euler

Task parameters:

- $dt = 0.05$
- MPC horizon $N = 80$ steps
- we sample K trajectories at each dt
- each control samples has the range

$$v = [-1.2, 1.2]$$
$$w = [-2.0, 2.0]$$



Algorithm 1 Generic iteration of Random Sampling

At each control loop:

- | | |
|---|--------------|
| 1: $\theta^{k=1,\dots,K} \sim p(\theta)$ | ▷ Sampling |
| 2: $J_k = \text{CostEval}(\theta^k)$ | ▷ Evaluation |
| 3: $\theta^* \leftarrow \text{select}(\theta^k, J^k)$ | ▷ Selection |
| 4: $\text{SendToMotors}(\theta^*)$ | ▷ Control |
-

We usually are in an MPC scenario, so we need to look ahead **N steps**!



$$\theta^i = [v_0, w_0, v_1, w_1, \dots, v_N, w_N]$$

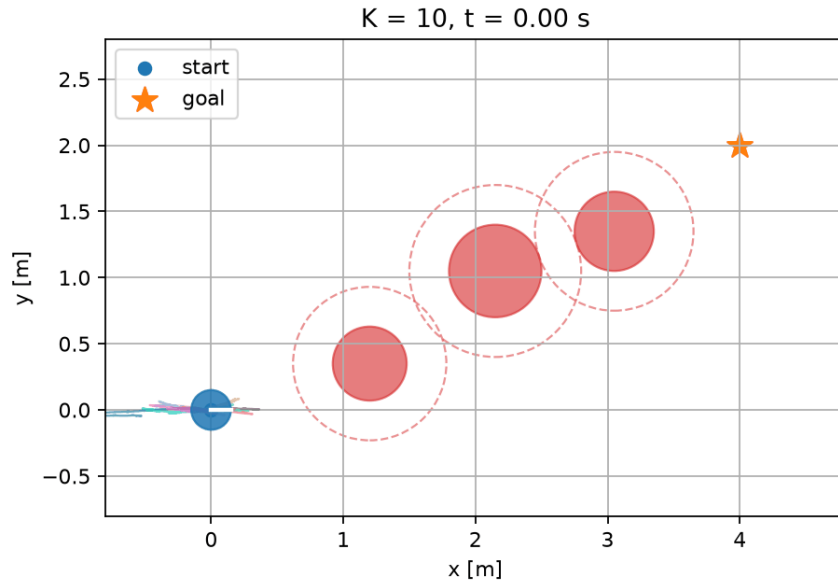
If u has dimension M ,
we **sample $K \times N \times M$ numbers**!

..or for simplicity, we say “**K trajectories**”

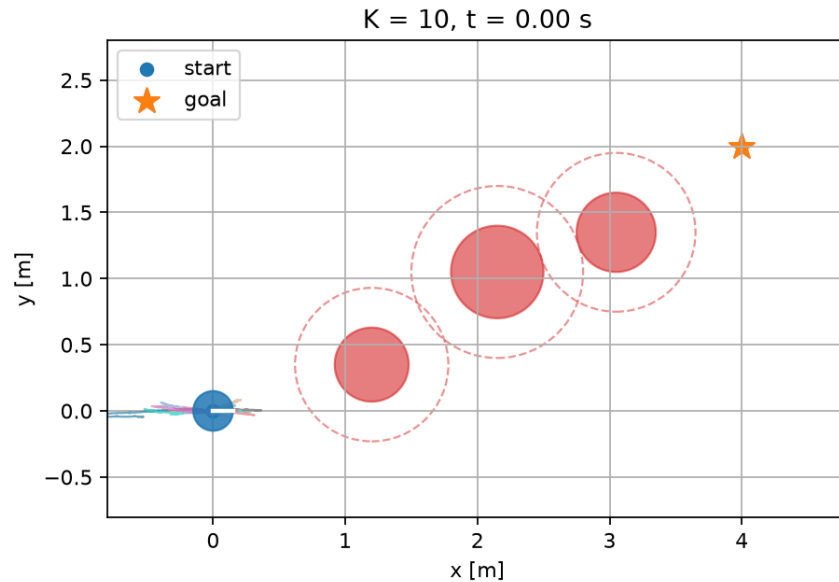
10 sample traj – not converge



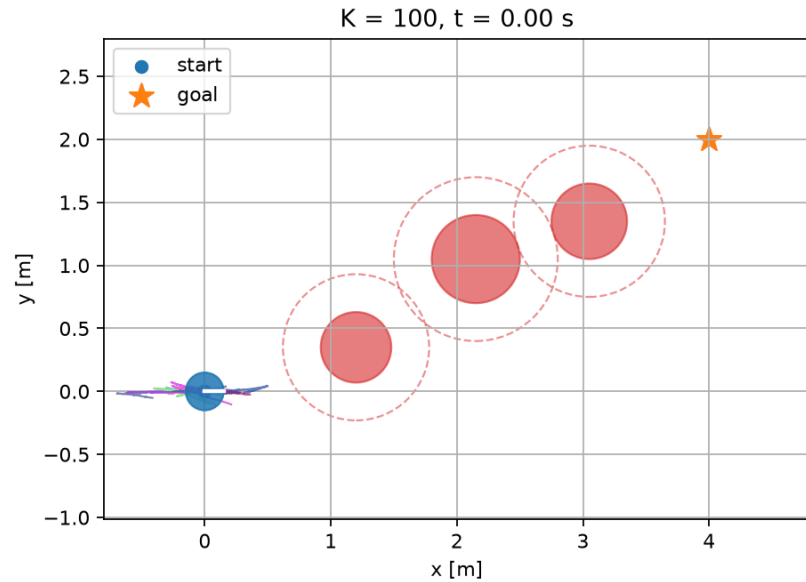
1600 sampled controls!



10 sample traj – not converge

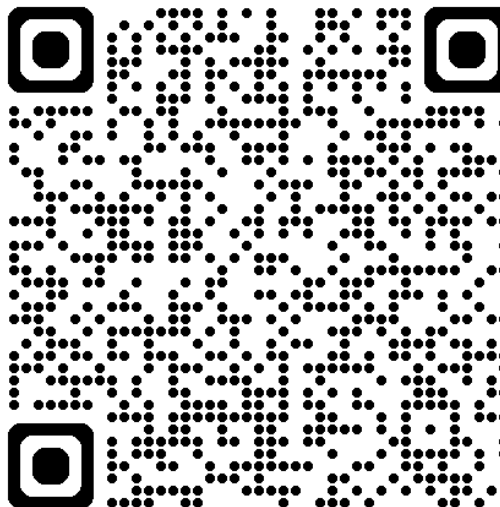


100 sample traj. – jittering



Link 1: Random Sampling

[https://colab.research.google.com/drive/1TW2pGfWtWyXf6tXc2dVlkl
yk3xpk0D5F?usp=sharing](https://colab.research.google.com/drive/1TW2pGfWtWyXf6tXc2dVlkl
yk3xpk0D5F?usp=sharing)

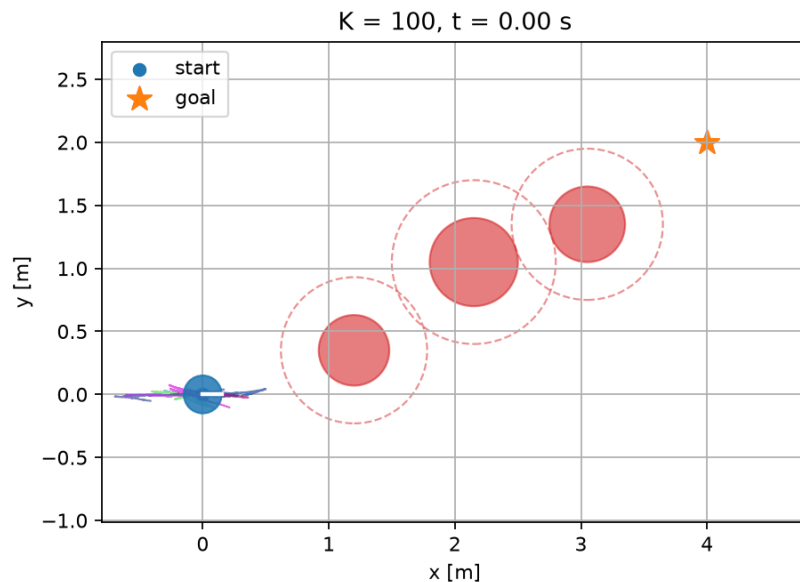


Half of the trajectory are not useful!

To increase optimality:

- bigger samples size
- warm-start
- better action space

100 sample traj. – jittering

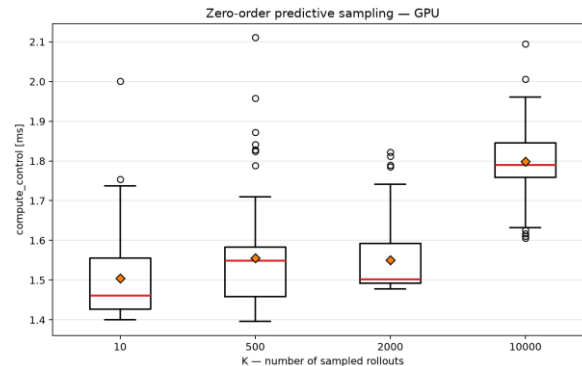
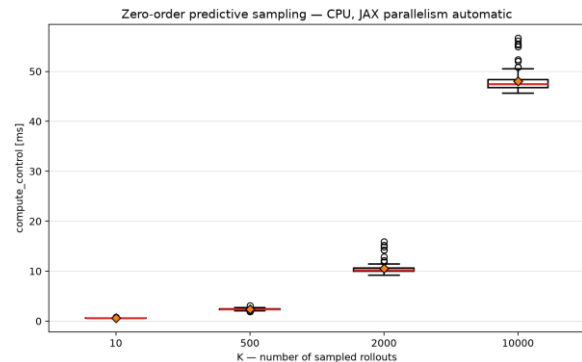


- Random sampling control
- **Practical limitations**
 - Computing
 - Warm-start
 - Splines
- CEM
- MPPI
- State-of-the-Art sampling in Legged Robots

Bigger sample size means better actions, but at the cost of computational time!

Considerations:

- Under 20ms is usually acceptable
- **CPU parallelism is essential**
(modern CPUs have >18cores)
- **For larger sample size, GPU are even more essential**
(modern GPUs have >1000 slower cores)



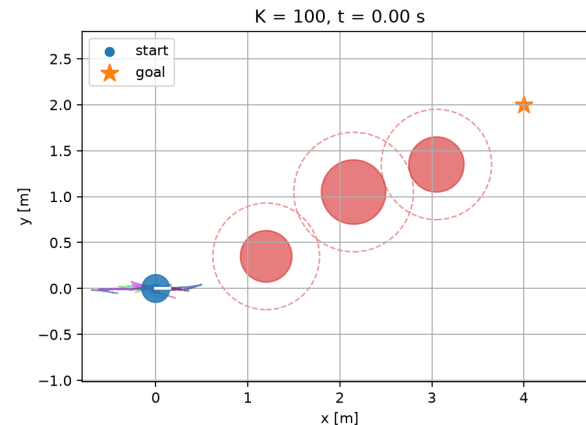
- every time we sample from these bounds $v = [-1.2, 1.2]$, $w = [-2.0, 2.0]$

```
 $\theta = \text{np.random.uniform}(-\text{bounds}, \text{bounds}, \text{size}=K)$ 
```

- Some trajectories are going backwards, but **at the previous iteration**, we found a better direction
- Idea: instead of sampling around zero, sample around the previous solution!

```
 $\Delta\theta^t = \text{np.random.uniform}(-\text{bounds}, \text{bounds}, \text{size}=K)$ 
```

```
 $\theta = \theta^{*,t} + \Delta\theta^t$ 
```



- Idea: instead of sampling around zero, sample around the previous solution!

```
 $\Delta\theta = \text{np.random.uniform}(-\text{bounds}, \text{bounds}, \text{size}=K)$   
 $\theta = \theta^{*,t} + \Delta\theta$ 
```

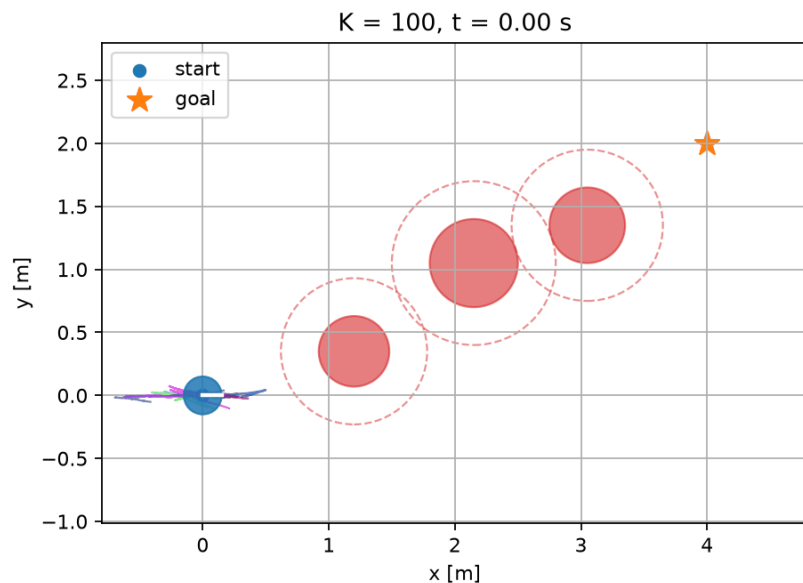


If the previous solution was good, the next one will be **in its neighborhood!**
we can restrict the search bounds!

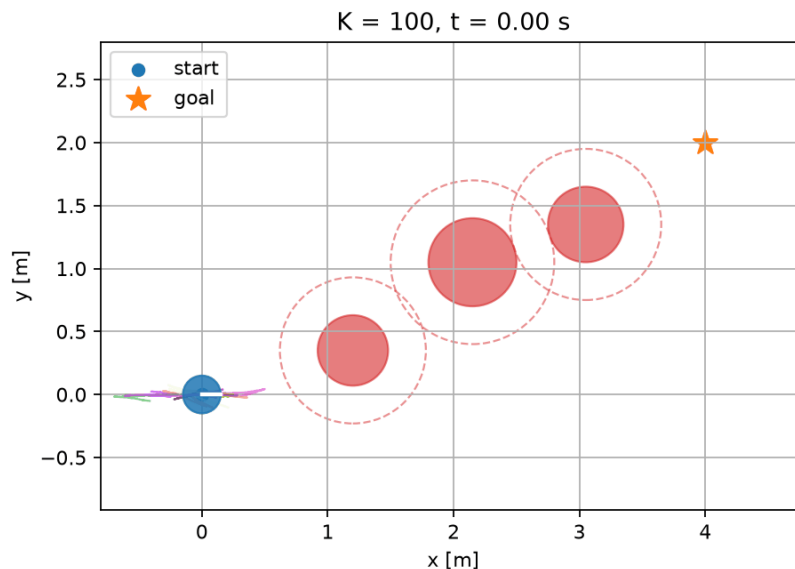
$$v = [-0.6, 0.6]$$

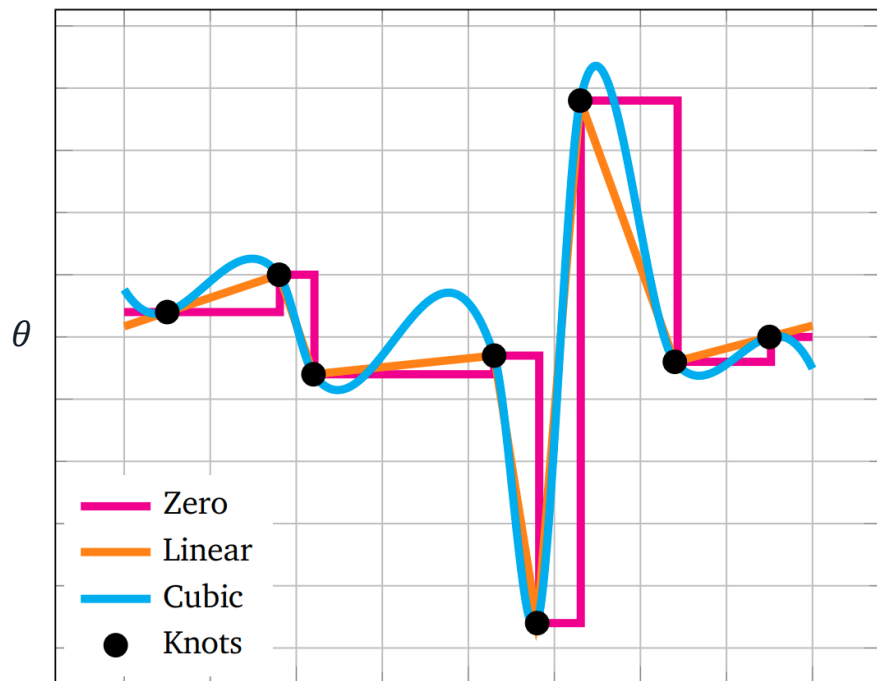
$$w = [-1.0, 1.0]$$

100 sample traj – jittering



100 sample traj, warm-start – better

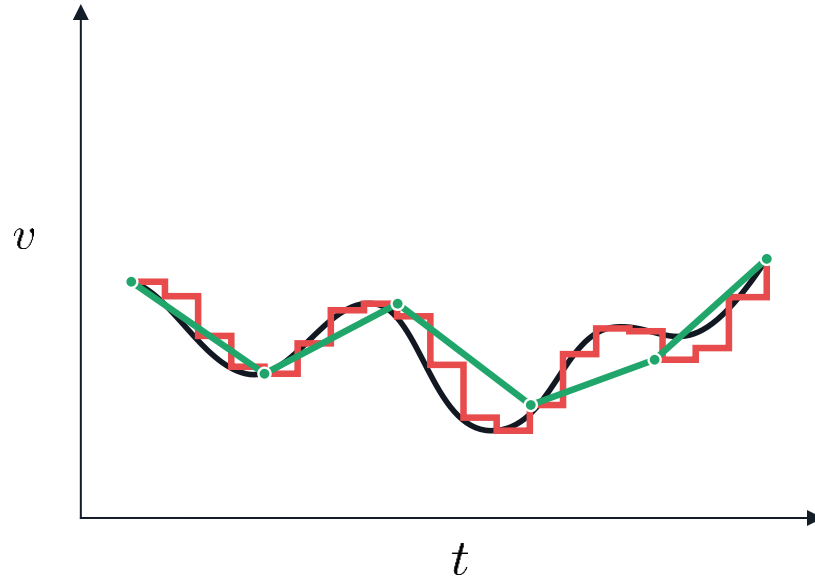




we can parametrize the actions as spline (**cubic** or **linear**) to reduce sample dimension

For a differential drive, with 2 commands and horizon N , we have:

- Zero-order = $2*N$
- **Linear** and **Cubic** representations can lead to a **smaller input space**, plus they ensure **smoothness**



For example, let's choose for our MPC problem:

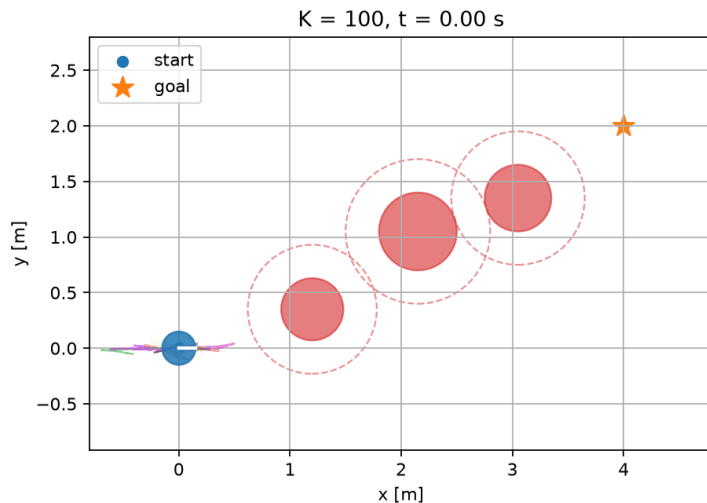
- $N=30$
- $dt = 0.01$

Zero-order dimension (single input): 30

Linear-spline dimension (single input): 6

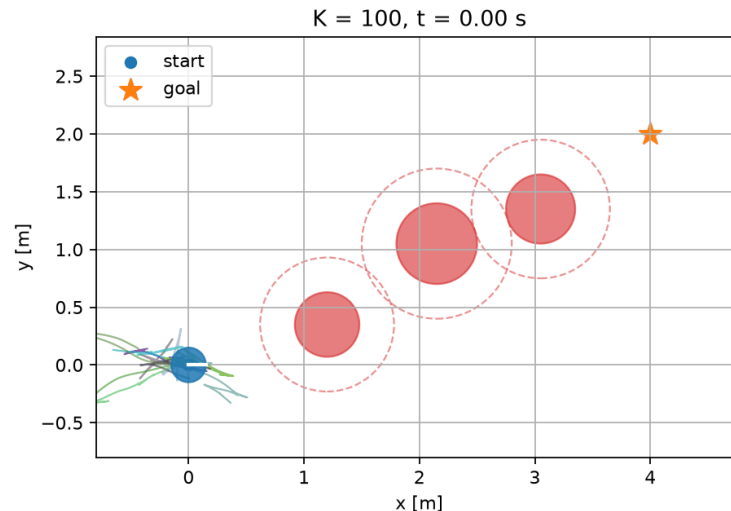
More “knots” means better approximation, but slower computation

100 sample traj, warm-start



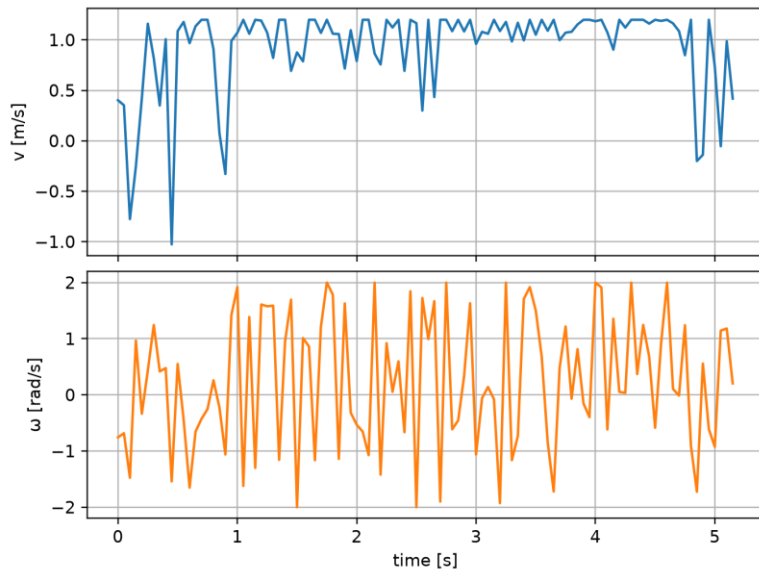
16000 random numbers sampled

linear spline, 20 knot -> 100 sample traj, warm-start

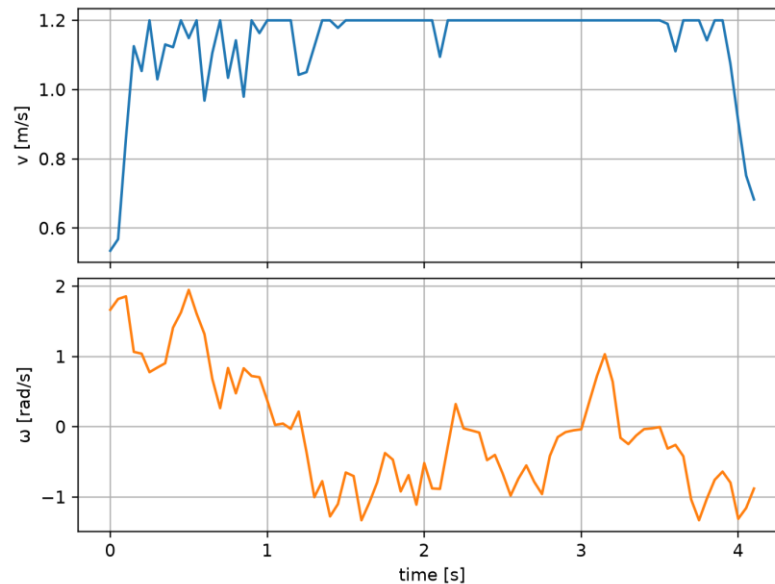


4000 random number sampled

100 sample traj, warm-start



linear spline, 20 knot -> 100 sample
traj, warm-start



- **without warm start**, we sampled between

$$v = [-1.2, 1.2] \quad w = [-2.0, 2.0]$$

- **with warm start**, we were able to sample between

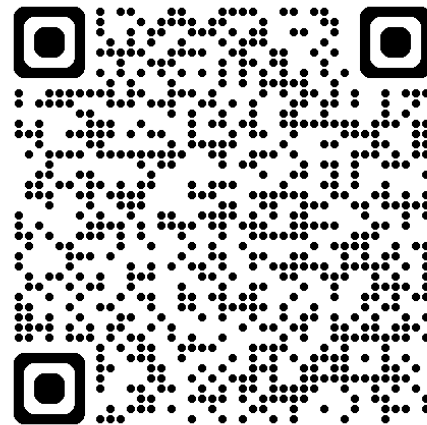
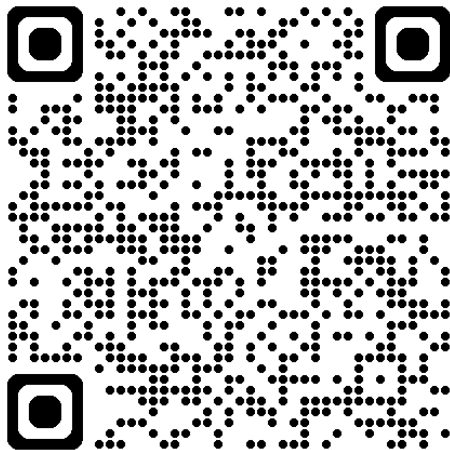
$$v = [-0.6, 0.6] \quad w = [-1.0, 1.0]$$

can we reduce the bounds even further?

- **Random Sampling** can be a good choice to optimize non-convex, mixed-integer problem
- PROS - very easy to implement
- CONS - no gradient info, meaning you need to increase the sample and/or reduce the search space
 - Warm start
 - GPU sampling
 - Splines

Link 2: Warm-Start

<https://colab.research.google.com/drive/1gZv1jEg23EIUn18GQ9un1deHDPISa8jN?usp=sharing>

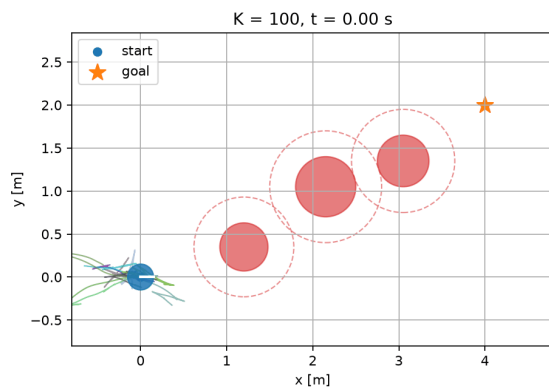


Link 3: Splines

<https://colab.research.google.com/drive/1Ql3Fj9OQLmhgAeC5ckYBoRCQYkXBtp9V?usp=sharing>

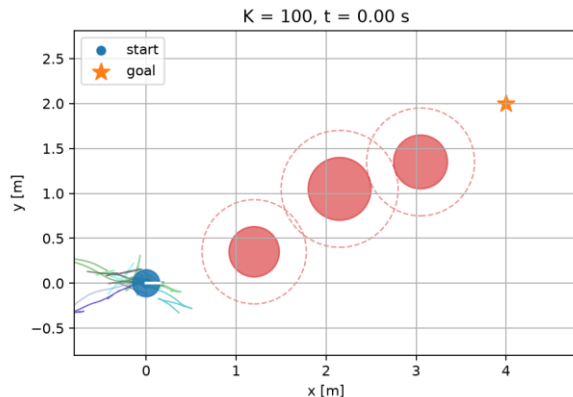
$$v = [-0.6, 0.6]$$

$$w = [-1.0, 1.0]$$



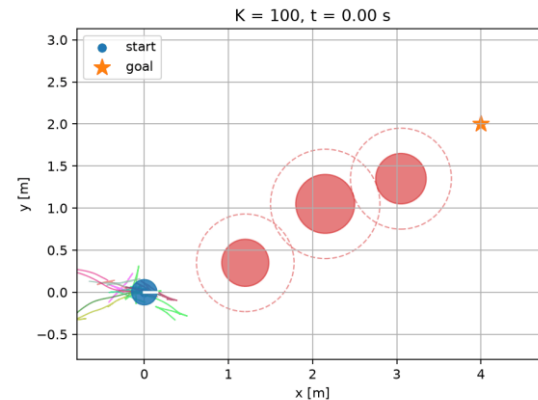
$$v = [-0.25, 0.25]$$

$$w = [-0.5, 0.5]$$



$$v = [-0.05, 0.05]$$

$$w = [-0.05, 0.05]$$



Different bounds, different performances. Smaller bounds can limit optimality.

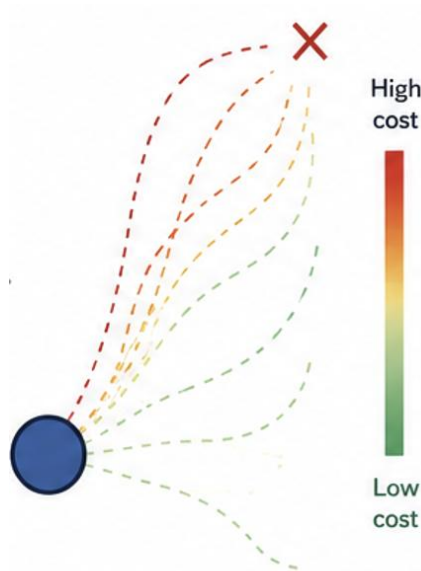
How do we choose these bounds?

- Random sampling control
- Practical limitations
- **CEM**
 - How to choose the sampling bounds?
 - Limitations
- State-of-the-Art sampling in Legged Robots

Idea:

Maybe we can find a new bounds, based on some previous top trajectories!

...If they are really closed, it means that **the solution is likely in between them!**

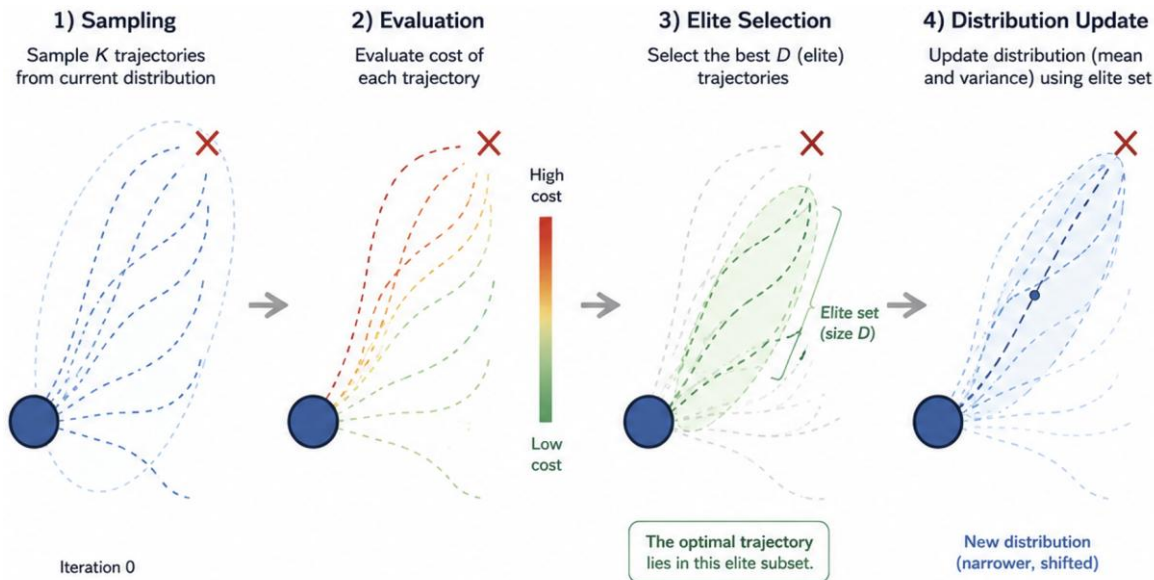


The best trajectory is likely closer to the red ones

Idea:

Maybe we can find a new bounds, based on some previous top trajectories!

...If they are really closed, it means that **the solution is likely in between them!**



Idea:

Maybe we can find a new bounds, based on some previous top trajectories!

...If they are really closed, it means that **the solution is likely in between them!**

Algorithm 1 Generic iteration of Random Sampling

At each control loop:

- 1: $\theta^{k=1,\dots,K} \sim p(\theta)$ ▷ Sampling
 - 2: $J_k = \text{CostEval}(\theta^k)$ ▷ Evaluation
 - 3: $\theta^* \leftarrow \text{select}(\theta^k, J^k)$ ▷ Selection
 - 4: $\text{SendToMotors}(\theta^*)$ ▷ Control
-

Algorithm 2 Cross-Entropy Method

At each control step:

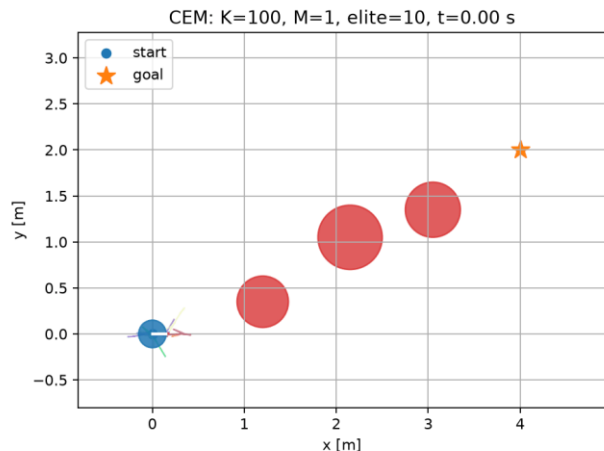
- 1: $\theta^{k=1,\dots,K} \sim p_p(\theta)$ ▷ Sampling
 - 2: $J_k = \text{CostEval}(\theta^k)$ ▷ Evaluation
 - 3: $\{\theta^{(1)}, \dots, \theta^{(D)}\} \leftarrow \text{selectBestD}(\theta^k, J_k)$ ▷ Selection
 - 4: $p_{p+1}(\theta) \leftarrow \text{UpdateRange}(\theta^{(1)}, \dots, \theta^{(D)})$ ▷ Update
 - 5: $\theta^* \leftarrow \theta^{(1)}$ ▷ Best trajectory
 - 6: $\text{SendToMotors}(\theta^*)$ ▷ Control
-

After some control steps, the sampling range collapse! **No more exploration**

Idea:

Maybe we can find a new bounds, based on some previous top trajectories!

...If they are really closed, it means that **the solution is likely in between them!**



Algorithm 2 Cross-Entropy Method

At each control step:

- 1: $\theta^{k=1,\dots,K} \sim p_p(\theta)$ ▷ Sampling
- 2: $J_k = \text{CostEval}(\theta^k)$ ▷ Evaluation
- 3: $\{\theta^{(1)}, \dots, \theta^{(D)}\} \leftarrow \text{selectBestD}(\theta^k, J_k)$ ▷ Selection
- 4: $p_{p+1}(\theta) \leftarrow \text{UpdateRange}(\theta^{(1)}, \dots, \theta^{(D)})$ ▷ Update
- 5: $\theta^* \leftarrow \theta^{(1)}$ ▷ Best trajectory
- 6: $\text{SendToMotors}(\theta^*)$ ▷ Control

After some control steps, the sampling range collapse! **No more exploration**

Problem:

After some control steps, the sampling range collapse! **No more exploration**

Solution:

- For each control step, do multiple iteration (like SQP)
- **Restart sampling range** in between control step to avoid exploration collapse!

Algorithm 2 Cross-Entropy Method

At each control step:

- 1: $\theta^{k=1,\dots,K} \sim p_p(\theta)$ ▷ Sampling
 - 2: $J_k = \text{CostEval}(\theta^k)$ ▷ Evaluation
 - 3: $\{\theta^{(1)}, \dots, \theta^{(D)}\} \leftarrow \text{selectBestD}(\theta^k, J_k)$ ▷ Selection
 - 4: $p_{p+1}(\theta) \leftarrow \text{UpdateRange}(\theta^{(1)}, \dots, \theta^{(D)})$ ▷ Update
 - 5: $\theta^* \leftarrow \theta^{(1)}$ ▷ Best trajectory
 - 6: $\text{SendToMotors}(\theta^*)$ ▷ Control
-

Algorithm 3 Cross-Entropy Method v2

At each control step:

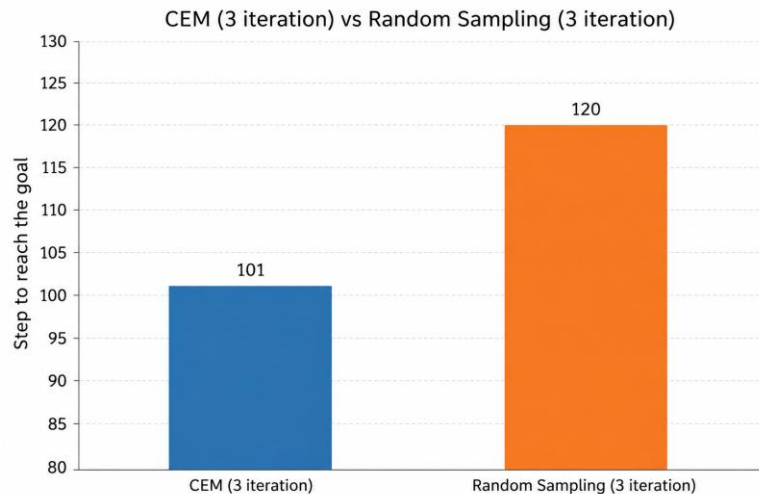
- 1: for $p = 0$ to P do
 - 2: $\theta^{k=1,\dots,K} \sim p_p(\theta)$ ▷ Sampling
 - 3: $J_k = \text{CostEval}(\theta^k)$ ▷ Evaluation
 - 4: $\{\theta^{(1)}, \dots, \theta^{(D)}\} \leftarrow \text{selectBestD}(\theta^k, J_k)$ ▷ Selection
 - 5: $p_{p+1}(\theta) \leftarrow \text{UpdateRange}(\theta^{(1)}, \dots, \theta^{(D)})$ ▷ Update
 - 6: end for
 - 7: $\theta^* \leftarrow \theta^{(1)}$ ▷ Best trajectory
 - 8: $p_0(\theta) \leftarrow p_{init}(\theta)$ ▷ Restore range
 - 9: $\text{SendToMotors}(\theta^*)$ ▷ Control
-

Problem:

After some control steps, the sampling range collapse! **No more exploration**

Solution:

- For each control step, do multiple iteration (like SQP)
- **Restart sampling range** in between control step to avoid exploration collapse!



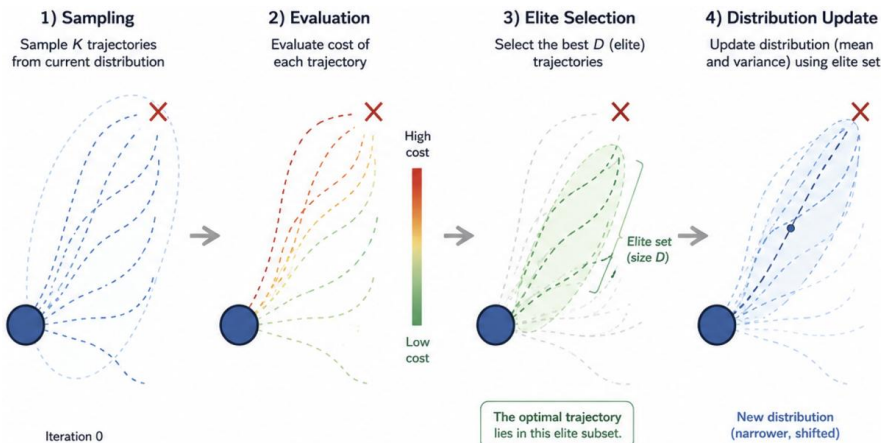
$$v = [-1.2, 1.2]$$

$$w = [-2.0, 2.0]$$

$$K = 100$$

- Random sampling control
- Practical limitations
- CEM
- **MPPI**
- State-of-the-Art sampling in Legged Robots

Let's find a new bounds, based on some previous top trajectories!
If they are really closed, it means that **the solution is likely in between them!**



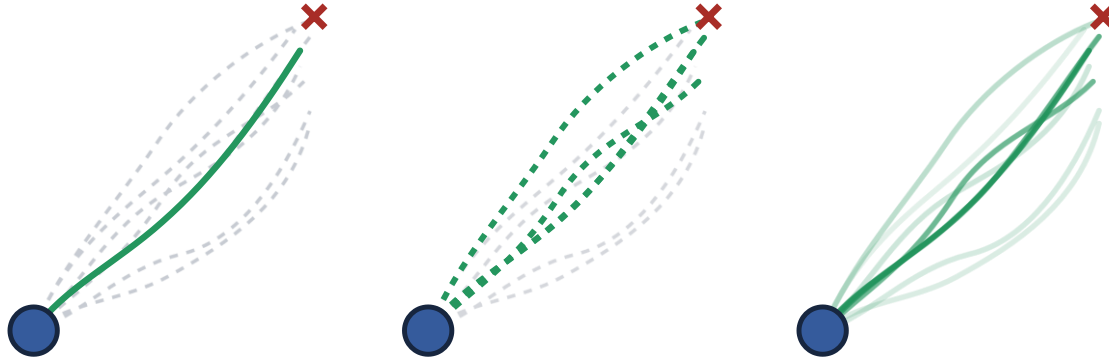
CEM only restrict the bound, but the choice of the best action is always:

$$\theta^* = \min(\theta, J)$$

Can we do better?

MPPI (Model Predictive Path Integral Control) build a smoothest actions, **weighting all the possible solution**

- **Random Sampling** discards all the solutions, **except one**
- **CEM** discards the majority of solutions, **except M elites**



MPPI (Model Predictive Path Integral Control) build a smoothest actions, **weighting all the possible solution**

Start nominal controls θ

for each control step:

- sample K noisy control sequences

- rollout dynamics

- compute trajectory costs

- compute weights

- update nominal control sequence

- apply first control of θ^*

MPPI (Model Predictive Path Integral Control) build a smoothest actions, **weighting all the possible solution**

- **Weights parameter** $w_k \propto \exp\left(-\frac{J_k}{\lambda}\right)$
- **All the samples' weights need to be sum to 1** $\sum_k w_k = 1$
- **Final formula** $w_k = \frac{\exp\left(-\frac{J_k}{\lambda}\right)}{\sum_j \exp\left(-\frac{J_j}{\lambda}\right)}$

MPPI (Model Predictive Path Integral Control) build a smoothest actions, **weighting all the possible solution**

- **Final formula**
$$w_k = \frac{\exp\left(-\frac{J_k}{\lambda}\right)}{\sum_j \exp\left(-\frac{J_j}{\lambda}\right)}$$
- **Control law:**
$$\theta_{*t} = \theta_{*t-1} + \sum_{k=1}^K w_k \Delta\theta_{t,k}$$

When MPPI is better than CEM?

- you avoid multiple iterations per single control step

Algorithm 3 Cross-Entropy Method v2

At each control step:

```

1: for  $p = 0$  to  $P$  do
2:    $\theta^{k=1,\dots,K} \sim p_p(\theta)$                                 ▷ Sampling
3:    $J_k = \text{CostEval}(\theta^k)$                                     ▷ Evaluation
4:    $\{\theta^{(1)}, \dots, \theta^{(D)}\} \leftarrow \text{selectBestD}(\theta^k, J_k)$  ▷ Selection
5:    $p_{p+1}(\theta) \leftarrow \text{UpdateRange}(\theta^{(1)}, \dots, \theta^{(D)})$  ▷ Update
6: end for
7:  $\theta^* \leftarrow \theta^{(1)}$                                        ▷ Best trajectory
8:  $p_0(\theta) \leftarrow p_{\text{init}}(\theta)$                          ▷ Restore range
9: SendToMotors( $\theta^*$ )                                         ▷ Control
  
```

- you don't know a-priori how many elites to use: MPPI uses all samples!

You can mix MPPI together with CEM

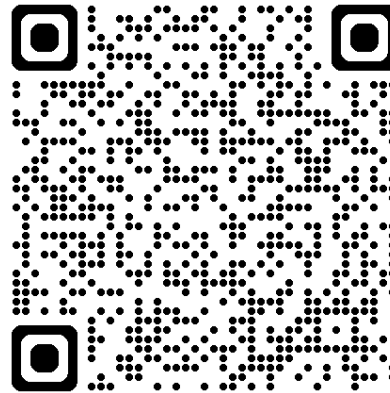


G. Williams et al., "Aggressive driving with model predictive path integral control", 2016 IEEE International Conference on Robotics and Automation (ICRA), 2016

- **Random Sampling** – how to choose the bound?
 - CEM estimate it from elites, but it can collapse. Remember to reset it from time to time!
- **Random Sampling** and **CEM** still discard the majority of samples
 - MPPI weights them all, and provides a smoother and more informative actions

Link 4: CEM

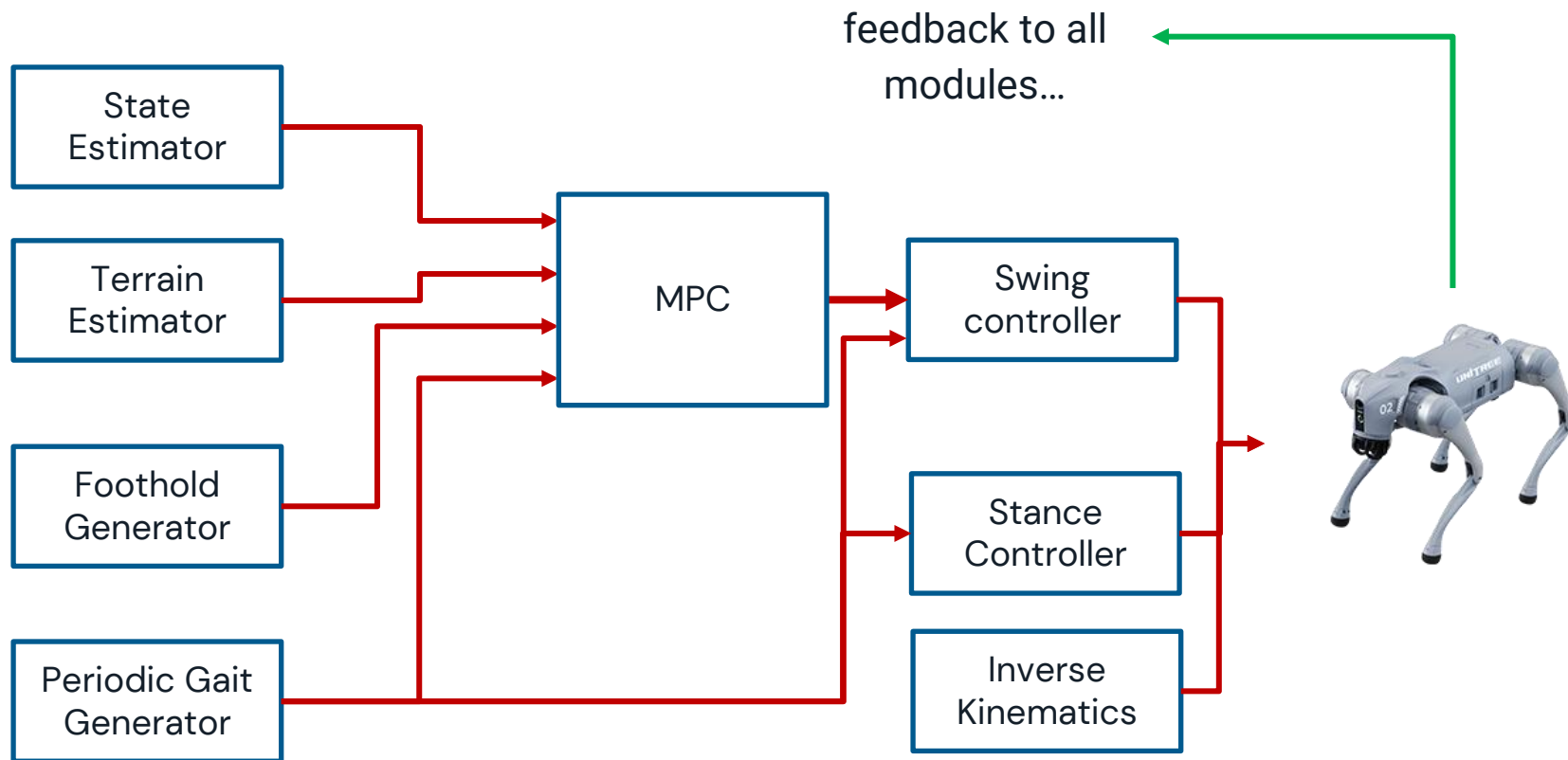
<https://colab.research.google.com/drive/1zbQMLud1hslLryBOUICXaR5JH-EdS5F2?usp=sharing>



- Random sampling control
- Practical limitations
- CEM
- MPPI
- **State-of-the-Art sampling in Legged Robots**

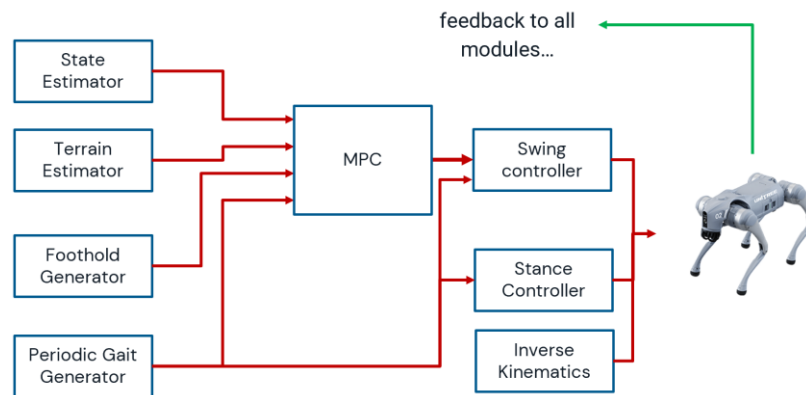
- Random sampling control
- Practical limitations
- CEM
- MPPI
- **State-of-the-Art sampling in Legged Robots**
 - Simplified Model
 - Full Model

General Model-Based Framework

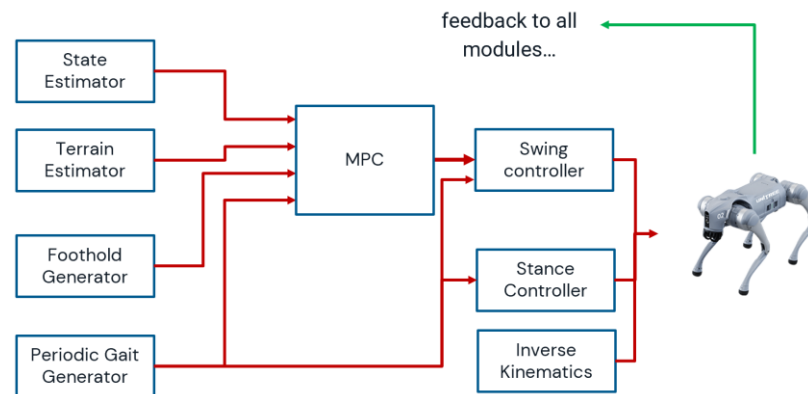


Kalman Filter which provides **state feedback**:

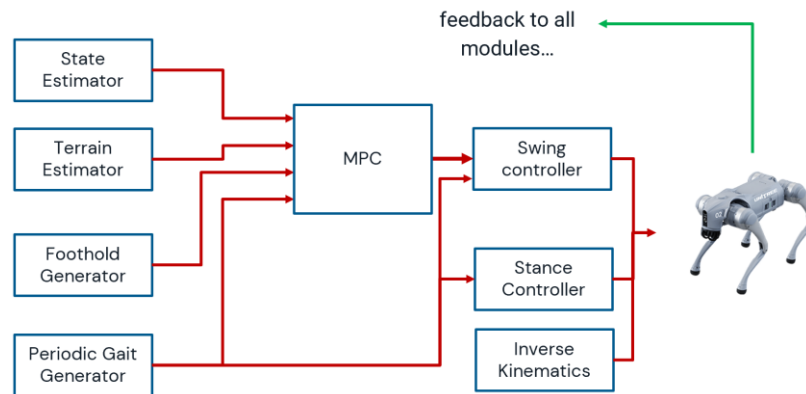
- base position
- base linear velocity
- base angular position
- base angular velocity



When the robot goes up the stairs, **how we should change its pitch and roll?**



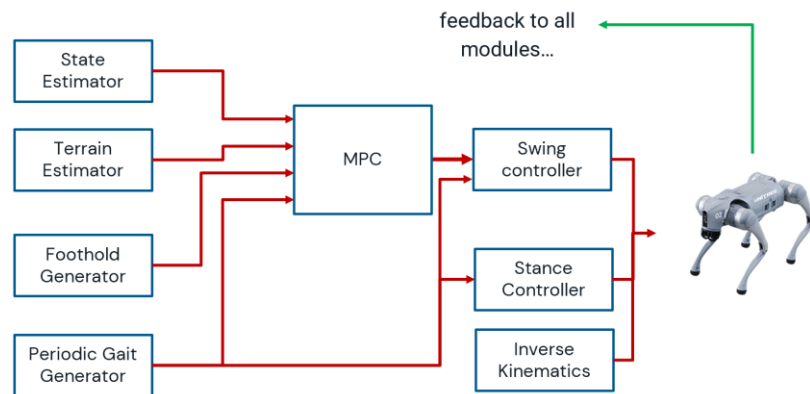
Where the robot should place **its feet**?



Periodic Gait Generator

Remember: model-based control usually precompute gait timing (e.g. when the foot should be on terrain – stance, and when in air - swing).

Which are the common gaits and how we generate them?



Trot



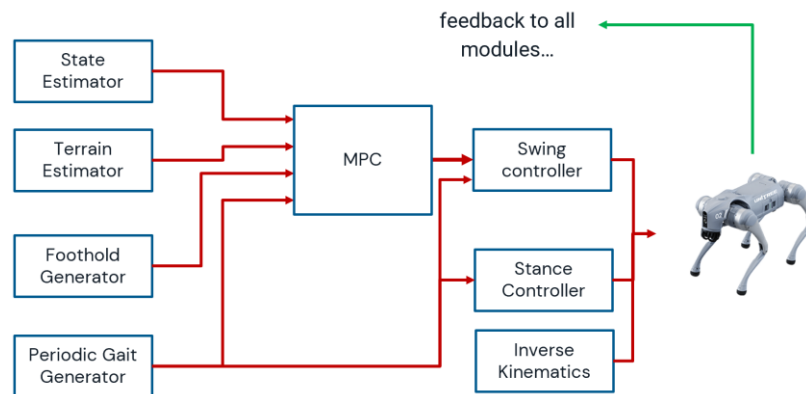
Crawl



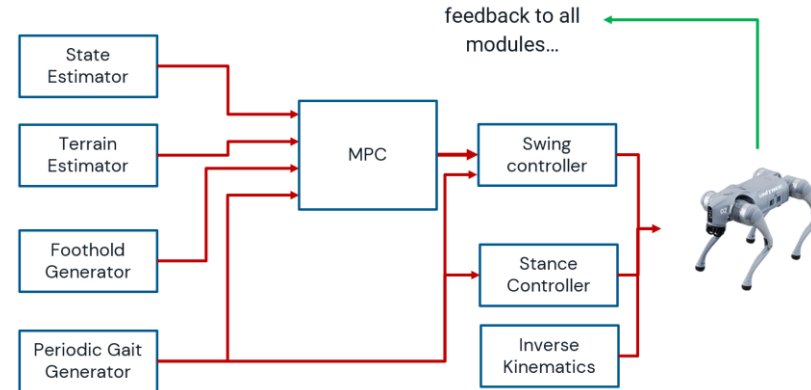
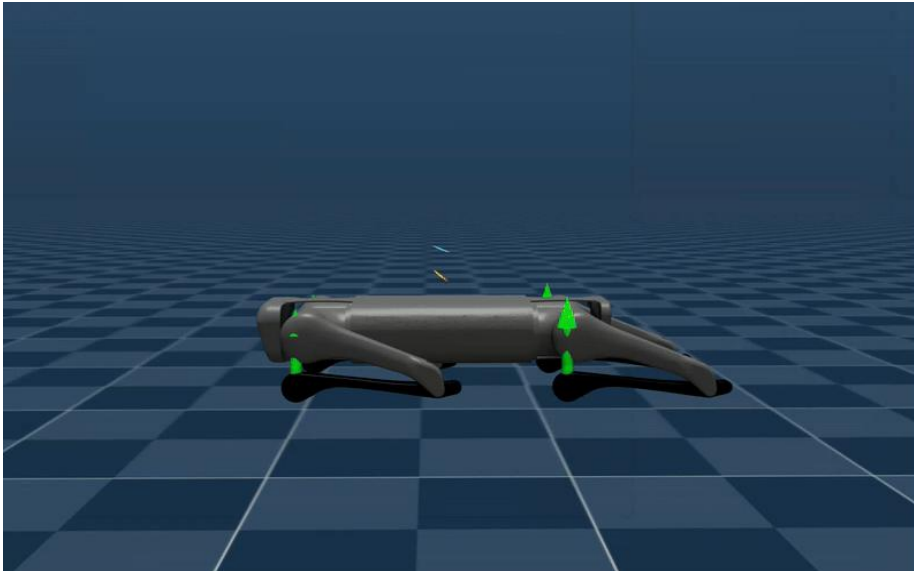
Pace

Based on:

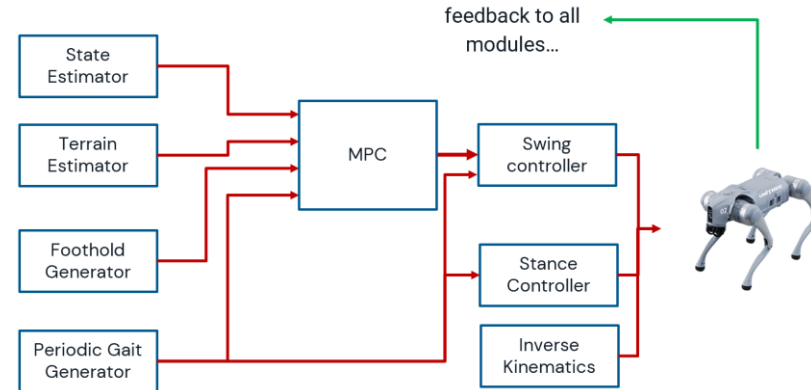
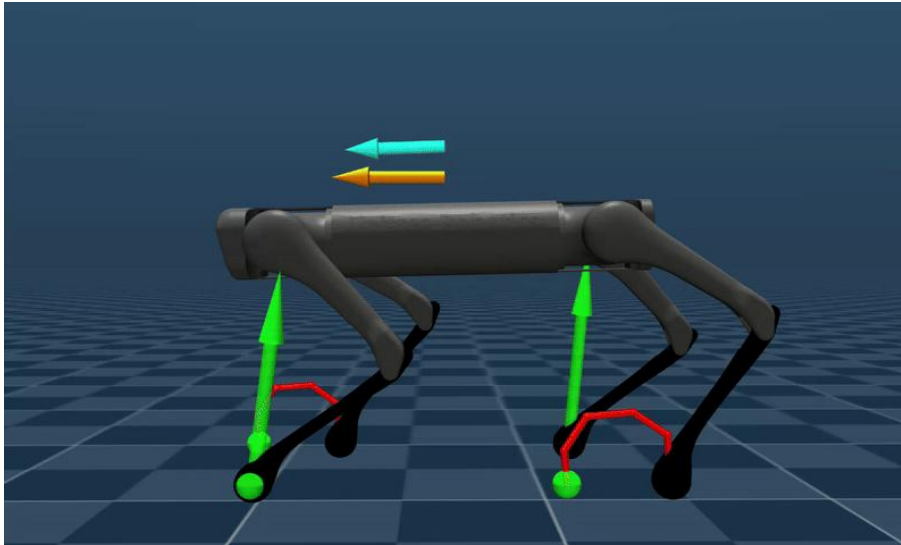
- LIP
- SRBD
-
- Full Model



When the MPC computes the GRF **for the feet in stance, how we map them into motor torques?**



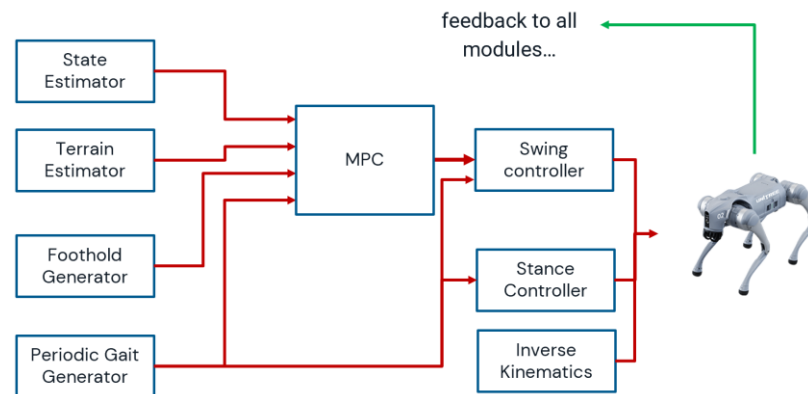
What happens instead to **the feet in air**?



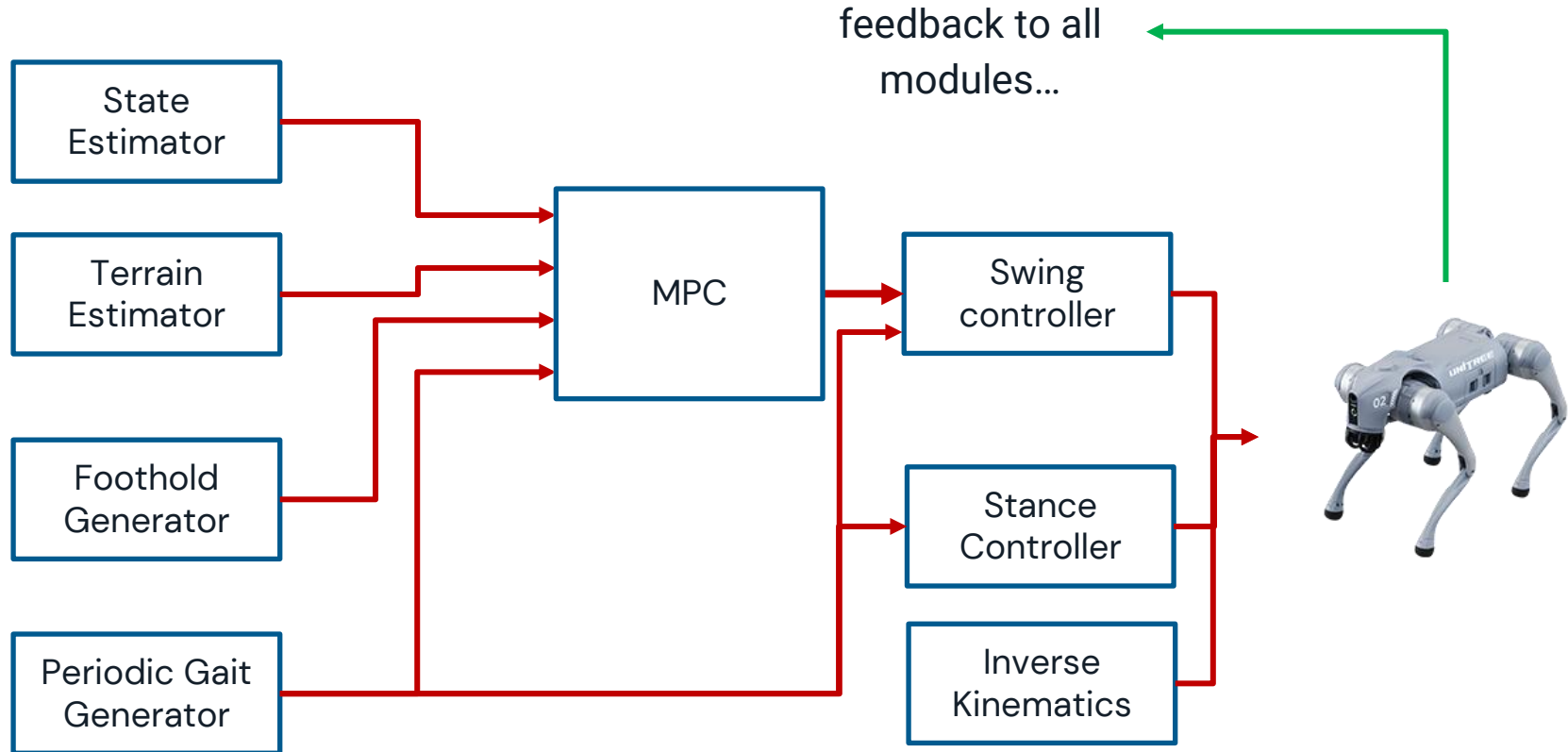
Real-robot have an internal closed loop, that is:

$$\tau_{ff} + K_p(q_{des} - q) + K_d(\dot{q}_{des} - \dot{q})$$

- τ_{ff} is called feed forward torque, came from swing and stance controller
- q_{des} doing **inverse kinematics**, asking “for a given foot position (e.g. in a particular swing moment), **which joints bring my foot there?**”



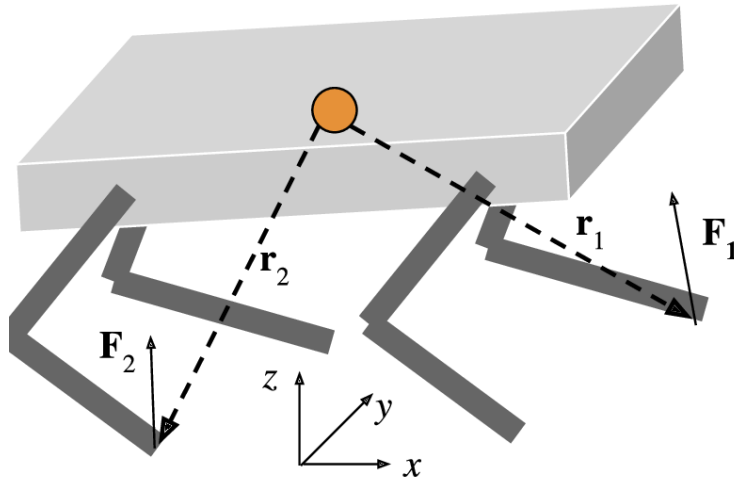
General Model-Based Framework



- Random sampling control
- Practical limitations
- CEM
- MPPI
- **State-of-the-Art sampling in Legged Robots**
 - Simplified Model
 - Full Model

SRBD model:

- only the robot's torso is considered
- inertial effects of leg movement are ignored
- can optimize foothold's location $\mathbf{r}$ together with Ground Reaction Forces $\mathbf{F}$

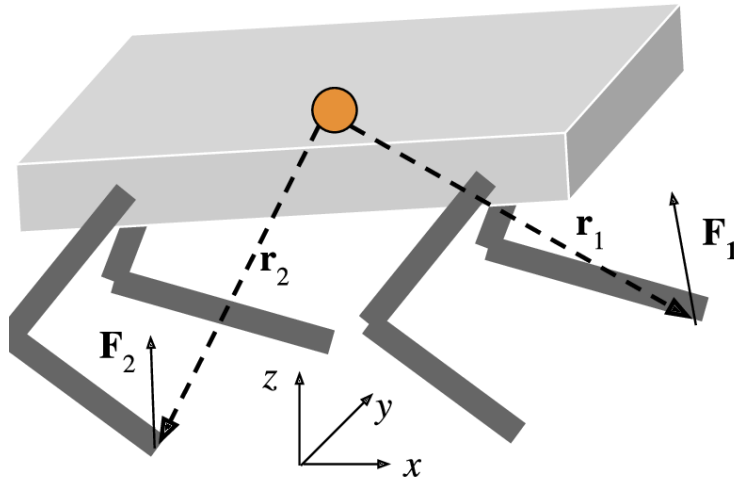


SRBD model:

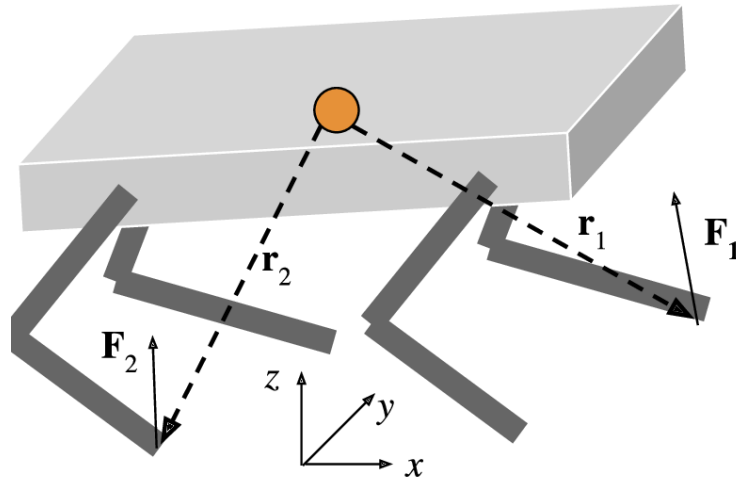
$$\mathbf{u} = [\mathbf{r}, \mathbf{F}, \delta] \quad \longrightarrow \text{inputs}$$

$$m(\ddot{x}_{com} + g) = \sum_{i=1}^c F_i \delta_i \quad \longrightarrow \text{linear part}$$

$$I_G \dot{\omega}_b \simeq \sum_{i=1}^c (F_i \times r_i) \quad \longrightarrow \text{angular part}$$



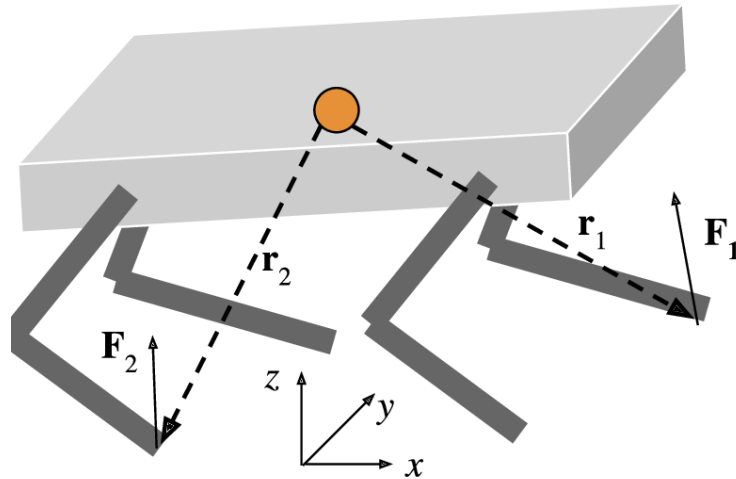
SRBD model:



$$m(\ddot{x}_{com} + g) = \sum_{i=1}^c F_i \delta_i \longrightarrow \text{linear part}$$

Delta is discrete $[0,1]$
(**mixed-integer program!**)

SRBD model:



$$I_G \dot{\omega}_b \simeq \sum_{i=1}^c (F_i \times r_i) \quad \longrightarrow \quad \text{angular part}$$

Bilinear product between r and F
(**non-convex!**)

- in gradient-based MPC, one usually **precompute** the gait sequence along the horizon beforehand



Trot

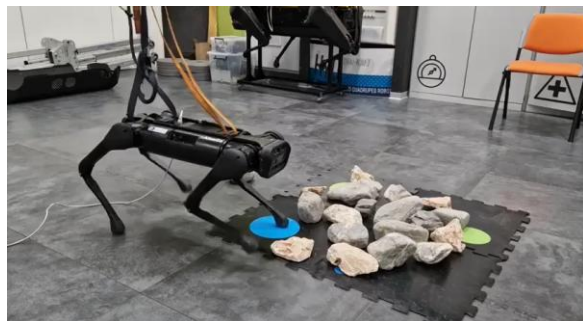


Crawl



Pace

- in gradient-based MPC, one usually **precompute** the gait sequence along the horizon beforehand



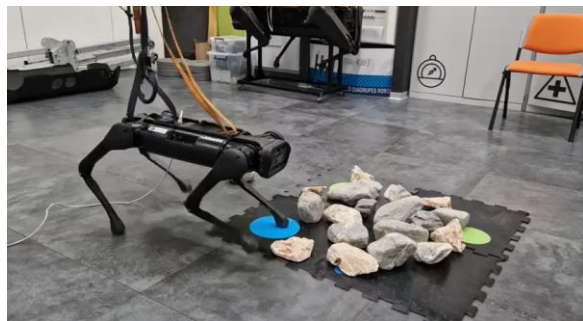
Trot

Front Left:	1 0 0 0 1 1 1 1 1
Front Right:	1 1 1 1 1 1 0 0 0
Rear Left:	1 1 1 1 1 1 0 0 0
Rear Right:	1 0 0 0 1 1 1 1 1

} gait sequence

- with an horizon N , we have $\delta_i = 16^N$ possible contacts! Too much to sample
- we can find a better parametrization

- in gradient-based MPC, one usually **precompute** the gait sequence along the horizon beforehand



Trot

Front Left:	1 0 0 0 1 1 1 1 1
Front Right:	1 1 1 1 1 1 0 0 0
Rear Left:	1 1 1 1 1 1 0 0 0
Rear Right:	1 0 0 0 1 1 1 1 1

gait sequence

f_s step frequency

D_f duty factor

$T_{st} = \frac{D_f}{f_s}$ stance time

$T_{sw} = \frac{1-D_f}{f_s}$ swing time

gait parameters
which we can
sample

Algorithm 5 SBS Predictive Controller for Locomotion

Given:

x_0 current state estimation

$\theta_{k=1\dots K} \sim \mathcal{N}(\theta, C)$

for each sample k (parallel GPU execution) do

$J_k = \text{Rollout}(\theta_k, x_0)$ ▷ See Algorithm 2

end for

$\theta_{k=1\dots K} \leftarrow \text{sort}(\theta_{k=1\dots K}, J_{k=1\dots K})$

$[\theta_1^{\text{new}}, \theta_2^{\text{new}}] = \text{UpdateMean}(\theta_{k=1\dots K_e}, J_{k=1\dots K_e})$

$C^{\text{new}} = C$

function $\pi(\theta, t_j)$

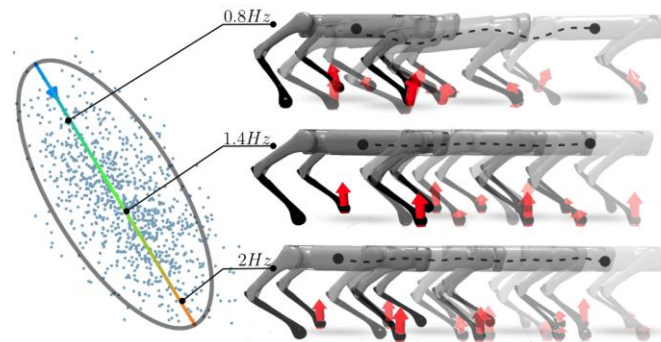
$[\theta_1, \theta_2] \leftarrow \theta$

$\delta_j \leftarrow \text{computeContactSequence}(\theta_1)$

$\Gamma_j \leftarrow \sigma((t_{1,\dots,P}^{\text{knot}}, \theta_{2,k}), t_j)$

$u_j = [\Gamma_j, \delta_j]$

end function



everything is implemented in **JAX**

Algorithm 5 SBS Predictive Controller for Locomotion

Given:

x_0 current state estimation

$\theta_{k=1\dots K} \sim \mathcal{N}(\theta, C)$

for each sample k (parallel GPU execution) **do**

$J_k = \text{Rollout}(\theta_k, x_0)$ ▷ See Algorithm 2

end for

$\theta_{k=1\dots K} \leftarrow \text{sort}(\theta_{k=1\dots K}, J_{k=1\dots K})$

$[\theta_1^{\text{new}}, \theta_2^{\text{new}}] = \text{UpdateMean}(\theta_{k=1\dots K_e}, J_{k=1\dots K_e})$

$C^{\text{new}} = C$

function $\pi(\theta, t_j)$

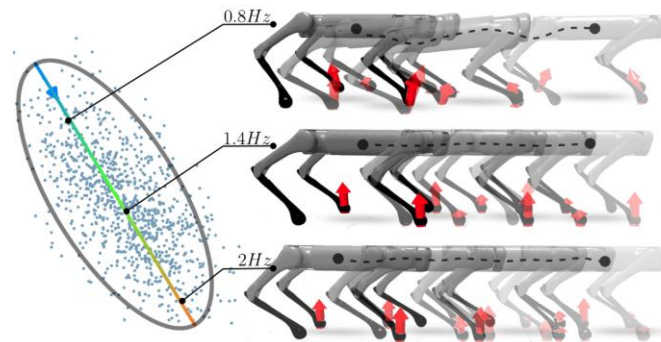
$[\theta_1, \theta_2] \leftarrow \theta$

$\delta_j \leftarrow \text{computeContactSequence}(\theta_1)$

$\Gamma_j \leftarrow \sigma((t_{1,\dots,P}^{\text{knot}}, \theta_{2,k}), t_j)$

$u_j = [\Gamma_j, \delta_j]$

end function



At each control loop:

- update the sensor information
- sample control actions

Algorithm 5 SBS Predictive Controller for Locomotion

Given:

x_0 current state estimation

$\theta_{k=1\dots K} \sim \mathcal{N}(\theta, C)$

for each sample k (parallel GPU execution) **do**

$J_k = \text{Rollout}(\theta_k, x_0)$ ▷ See Algorithm 2

end for

$\theta_{k=1\dots K} \leftarrow \text{sort}(\theta_{k=1\dots K}, J_{k=1\dots K})$

$[\theta_1^{\text{new}}, \theta_2^{\text{new}}] = \text{UpdateMean}(\theta_{k=1\dots K_e}, J_{k=1\dots K_e})$

$C^{\text{new}} = C$

function $\pi(\theta, t_j)$

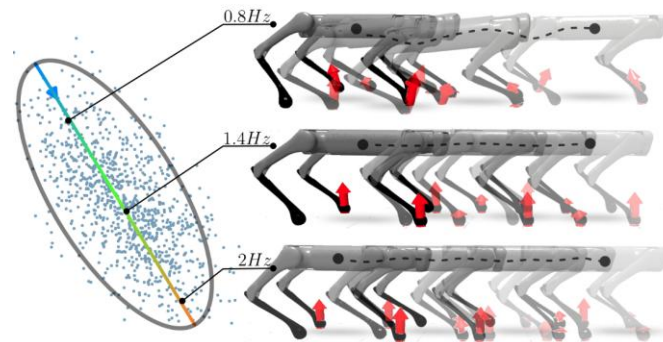
$[\theta_1, \theta_2] \leftarrow \theta$

$\delta_j \leftarrow \text{computeContactSequence}(\theta_1)$

$\Gamma_j \leftarrow \sigma((t_{1,\dots,P}^{\text{knot}}, \theta_{2,k}), t_j)$

$u_j = [\Gamma_j, \delta_j]$

end function



we optimize both ground reaction forces
and **gait parameters**

Algorithm 5 SBS Predictive Controller for Locomotion

Given:

x_0 current state estimation

$\theta_{k=1\dots K} \sim \mathcal{N}(\theta, C)$

for each sample k (parallel GPU execution) do

$J_k = \text{Rollout}(\theta_k, x_0)$ ▷ See Algorithm 2

end for

$\theta_{k=1\dots K} \leftarrow \text{sort}(\theta_{k=1\dots K}, J_{k=1\dots K})$

$[\theta_1^{\text{new}}, \theta_2^{\text{new}}] = \text{UpdateMean}(\theta_{k=1\dots K_e}, J_{k=1\dots K_e})$

$C^{\text{new}} = C$

function $\pi(\theta, t_j)$

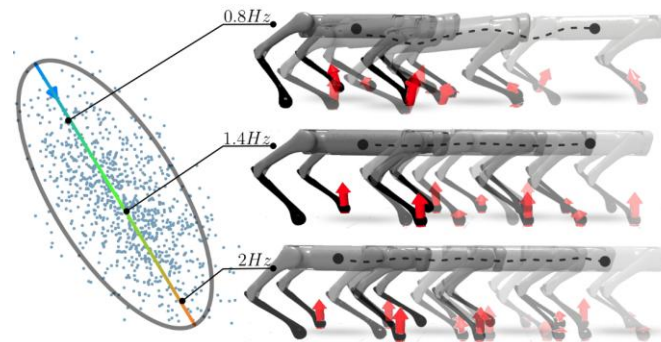
$[\theta_1, \theta_2] \leftarrow \theta$

$\delta_j \leftarrow \text{computeContactSequence}(\theta_1)$

$\Gamma_j \leftarrow \sigma((t_{1,\dots,P}^{\text{knot}}, \theta_{2,k}), t_j)$

$u_j = [\Gamma_j, \delta_j]$

end function



step frequency and duty factor

ground reaction forces with linear spline

Algorithm 5 SBS Predictive Controller for Locomotion

Given:

x_0 current state estimation

$\theta_{k=1\dots K} \sim \mathcal{N}(\theta, C)$

for each sample k (parallel GPU execution) do

$J_k = \text{Rollout}(\theta_k, x_0)$ ▷ See Algorithm 2

end for

$\theta_{k=1\dots K} \leftarrow \text{sort}(\theta_{k=1\dots K}, J_{k=1\dots K})$

$[\theta_1^{\text{new}}, \theta_2^{\text{new}}] = \text{UpdateMean}(\theta_{k=1\dots K_e}, J_{k=1\dots K_e})$

$C^{\text{new}} = C$

function $\pi(\theta, t_j)$

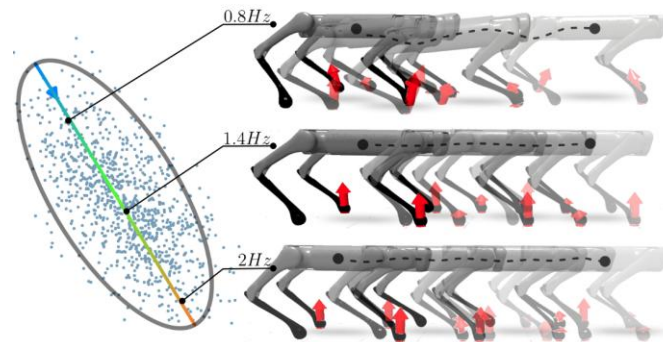
$[\theta_1, \theta_2] \leftarrow \theta$

$\delta_j \leftarrow \text{computeContactSequence}(\theta_1)$

$\Gamma_j \leftarrow \sigma((t_{1,\dots,P}^{\text{knot}}, \theta_{2,k}), t_j)$

$u_j = [\Gamma_j, \delta_j]$

end function



step frequency and duty factor

ground reaction forces with linear spline

Algorithm 5 SBS Predictive Controller for Locomotion

Given:

x_0 current state estimation

$\theta_{k=1\dots K} \sim \mathcal{N}(\theta, C)$

for each sample k (parallel GPU execution) do
 $J_k = \text{Rollout}(\theta_k, x_0)$ $\triangleright$ See Algorithm 2
end for

$\theta_{k=1\dots K} \leftarrow \text{sort}(\theta_{k=1\dots K}, J_{k=1\dots K})$

$[\theta_1^{\text{new}}, \theta_2^{\text{new}}] = \text{UpdateMean}(\theta_{k=1\dots K_e}, J_{k=1\dots K_e})$

$C^{\text{new}} = C$

function $\pi(\theta, t_j)$

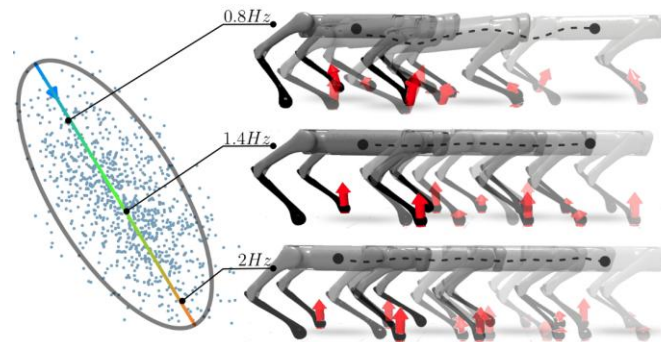
$[\theta_1, \theta_2] \leftarrow \theta$

$\delta_j \leftarrow \text{computeContactSequence}(\theta_1)$

$\Gamma_j \leftarrow \sigma((t_{1,\dots,P}^{\text{knot}}, \theta_{2,k}), t_j)$

$u_j = [\Gamma_j, \delta_j]$

end function



- **simulate** the centroidal model via Euler or RK4
- tracking cost + minimize step freq.

Algorithm 5 SBS Predictive Controller for Locomotion

Given:

x_0 current state estimation

$\theta_{k=1\dots K} \sim \mathcal{N}(\theta, C)$

for each sample k (parallel GPU execution) do

$J_k = \text{Rollout}(\theta_k, x_0)$ $\triangleright$ See Algorithm 2

end for

$\theta_{k=1\dots K} \leftarrow \text{sort}(\theta_{k=1\dots K}, J_{k=1\dots K})$

$[\theta_1^{\text{new}}, \theta_2^{\text{new}}] = \text{UpdateMean}(\theta_{k=1\dots K_e}, J_{k=1\dots K_e})$

$C^{\text{new}} = C$

function $\pi(\theta, t_j)$

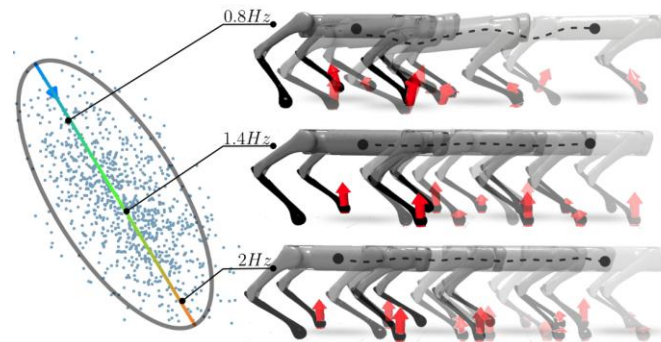
$[\theta_1, \theta_2] \leftarrow \theta$

$\delta_j \leftarrow \text{computeContactSequence}(\theta_1)$

$\Gamma_j \leftarrow \sigma((t_{1,\dots,P}^{\text{knot}}, \theta_{2,k}), t_j)$

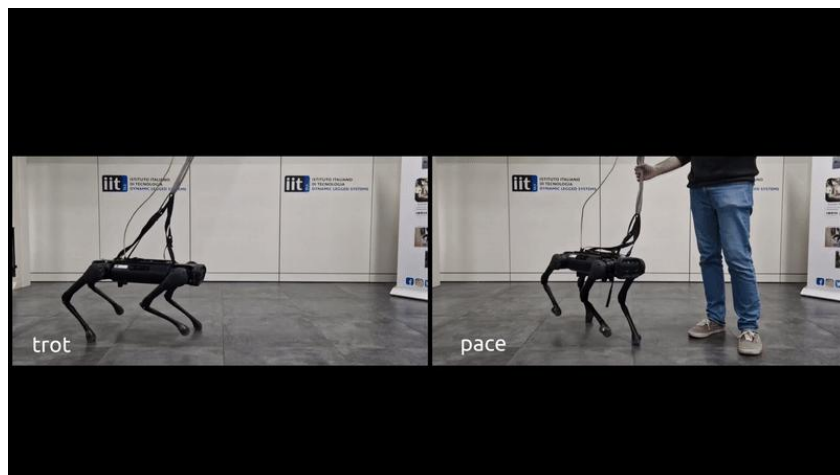
$u_j = [\Gamma_j, \delta_j]$

end function

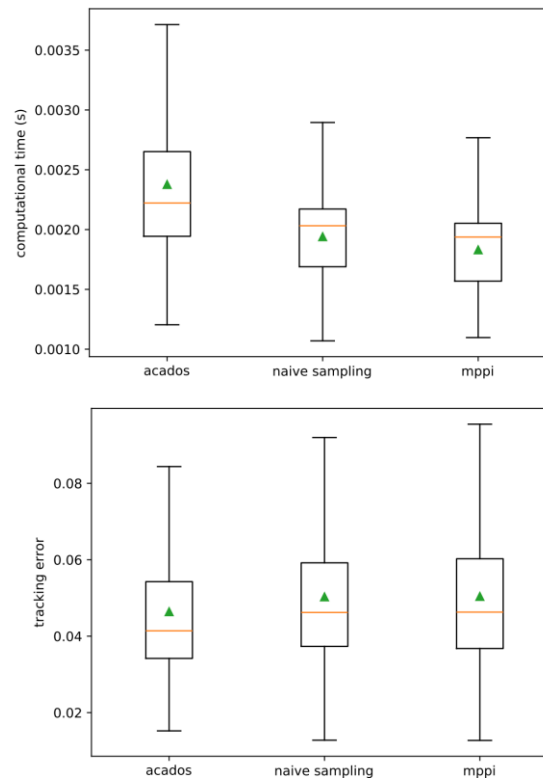


take the **best action** and repeat

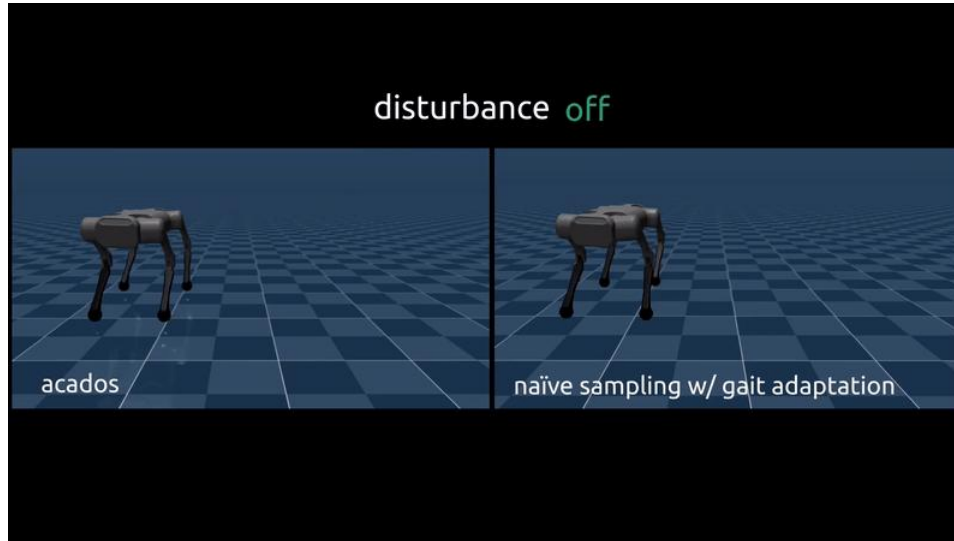
Simplified Quadruped Model



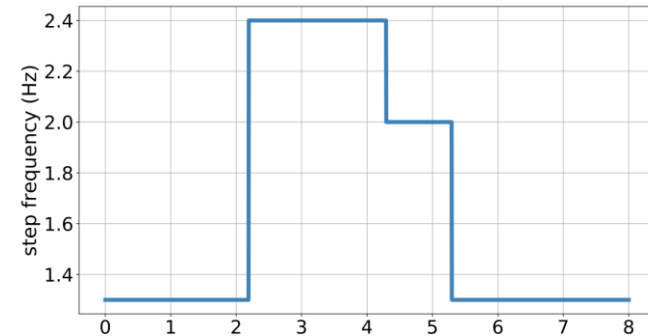
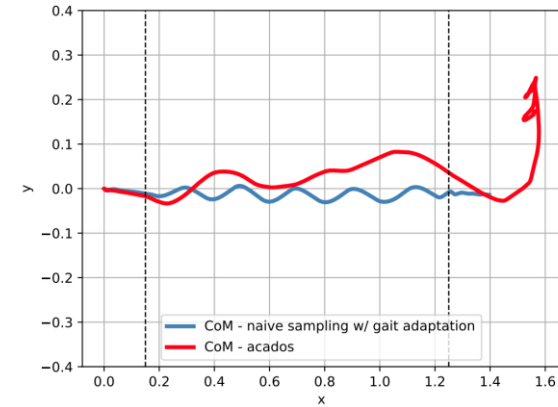
- different gaits can be easily performed
- same performances of gradient-based methods with an **incredibly simple control law**



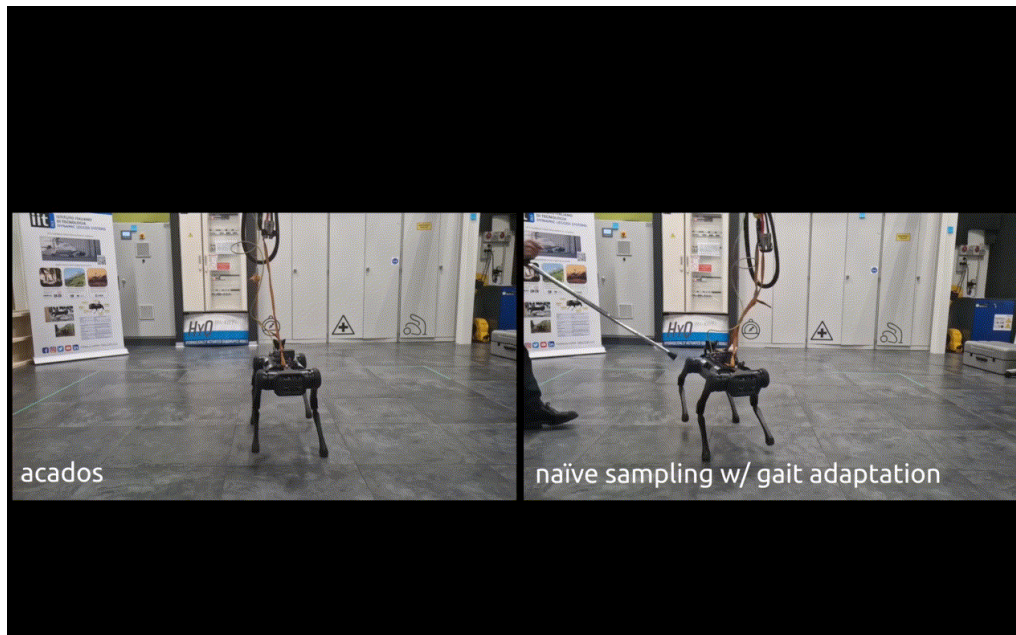
Simplified Quadruped Model



Automatic gait adaptation

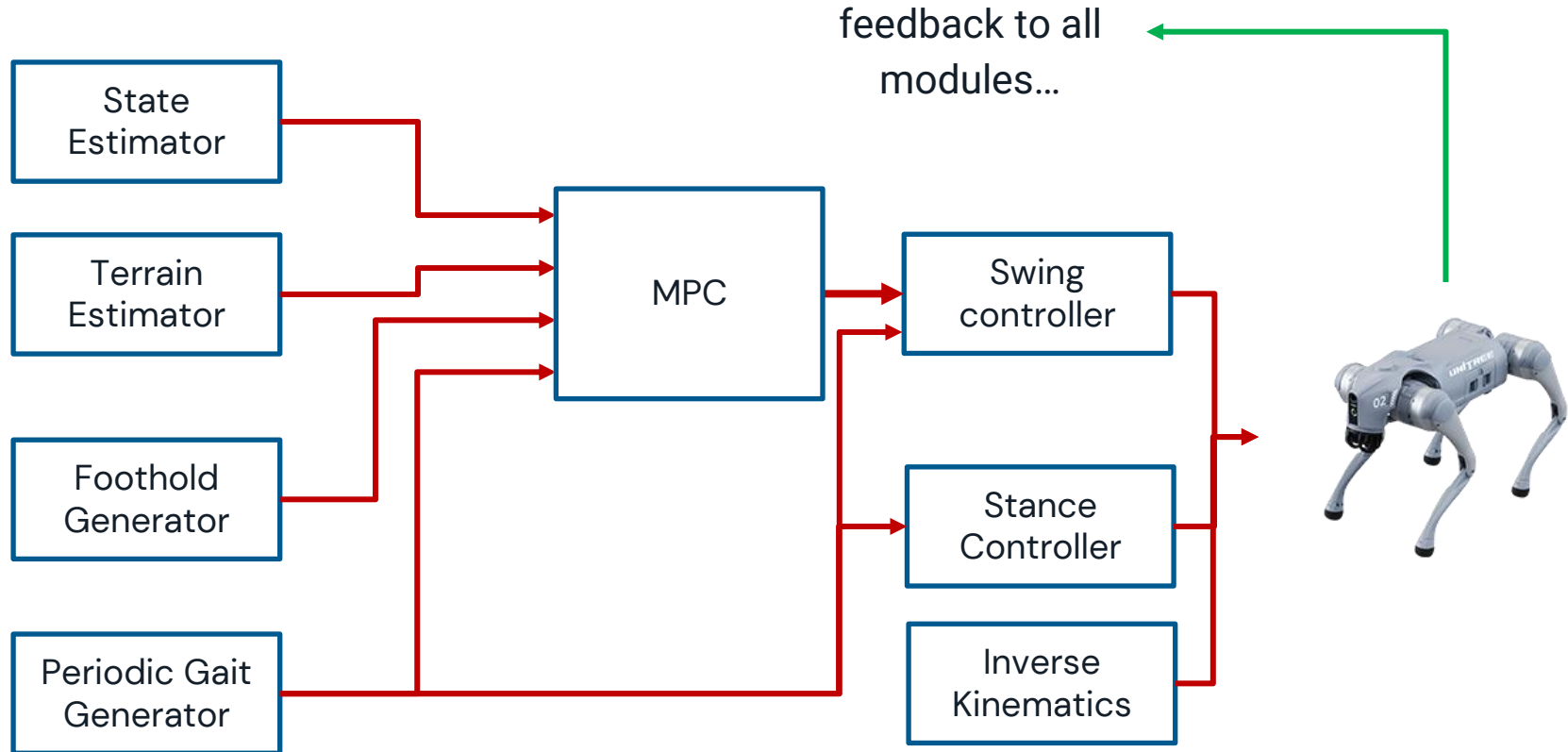


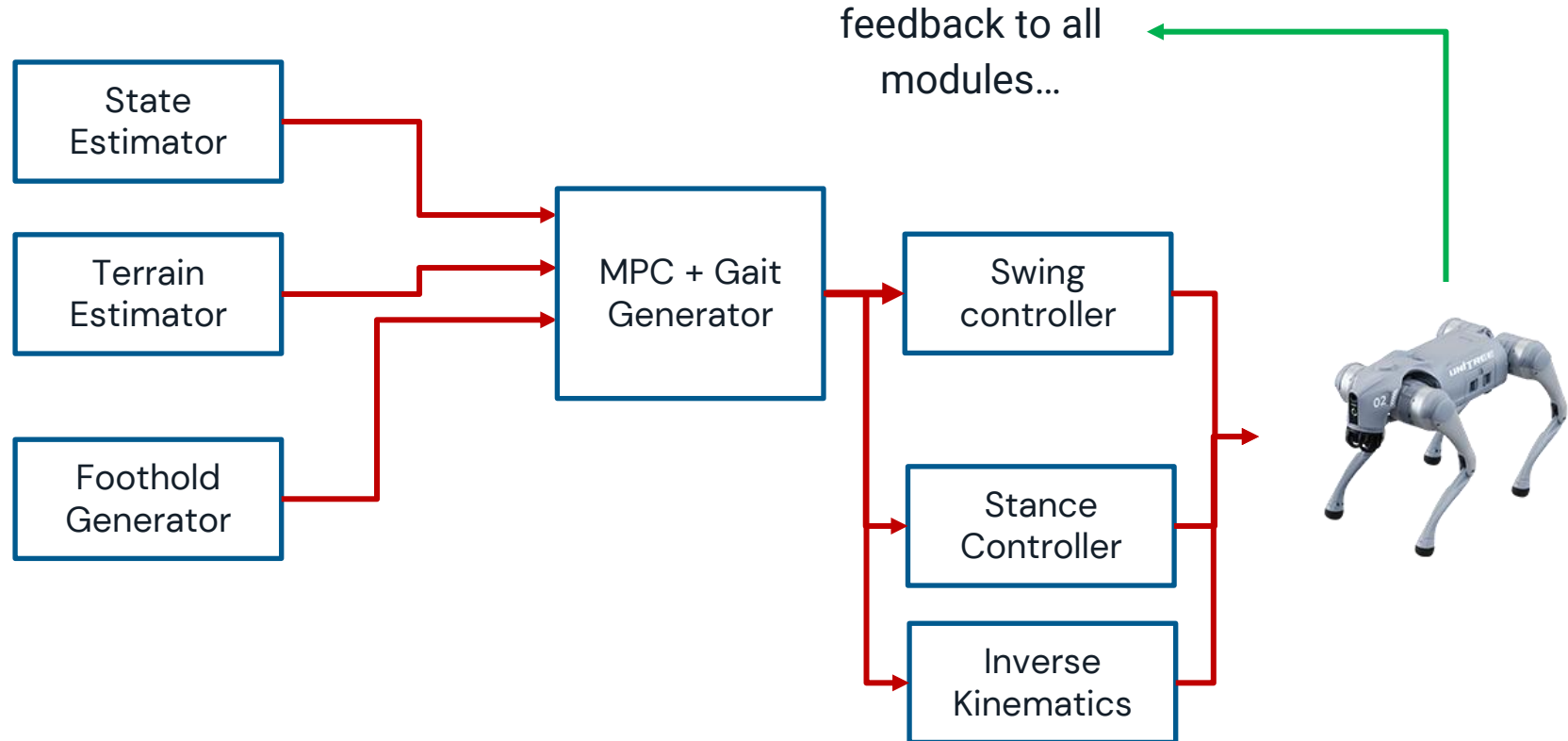
Simplified Quadruped Model



Higher stepping frequency at need to increase robustness

General Model-Based Framework

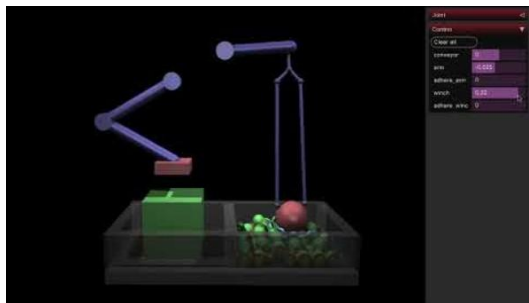


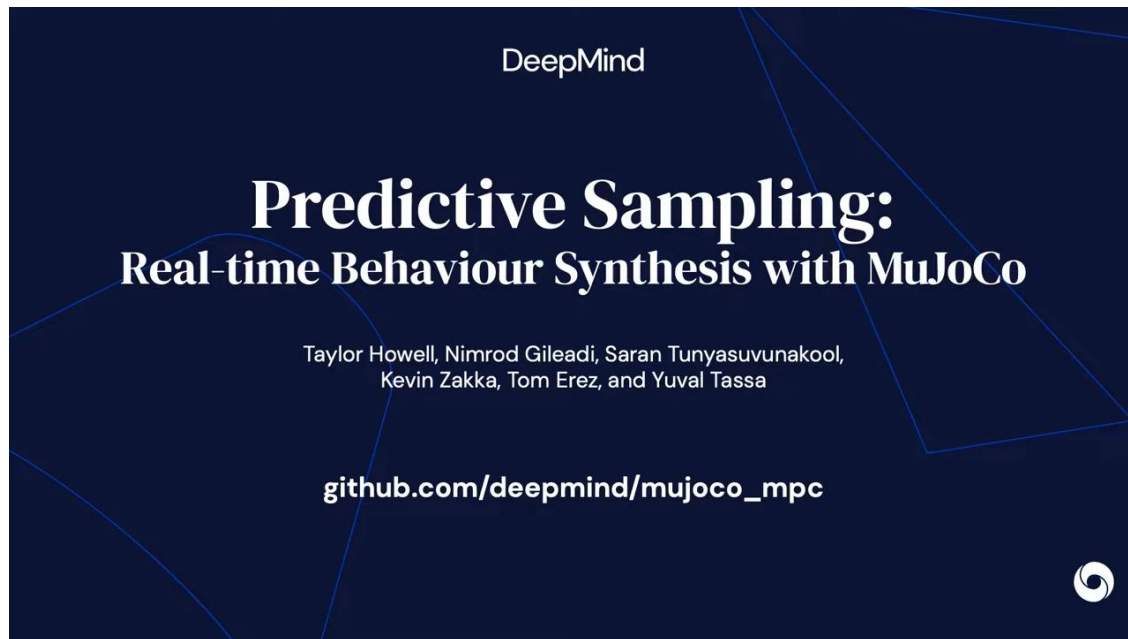


- Simplified model are perfect to reduce sampling space
- Can sampling work with **full rigid body dynamics**?

Mujoco Simulator

- Created by Emo Todorov, Tom Erez, Yuval Tassa (2012!)
- Recently acquired by Google Deepmind, Open Source
- One of the fastest CPU simulators out there (now in GPU as well, more tomorrow!)
- Soft contact modelling





DeepMind

Predictive Sampling: Real-time Behaviour Synthesis with MuJoCo

Taylor Howell, Nimrod Gileadi, Saran Tunyasuvunakool,
Kevin Zakka, Tom Erez, and Yuval Tassa

github.com/deepmind/mujoco_mpc

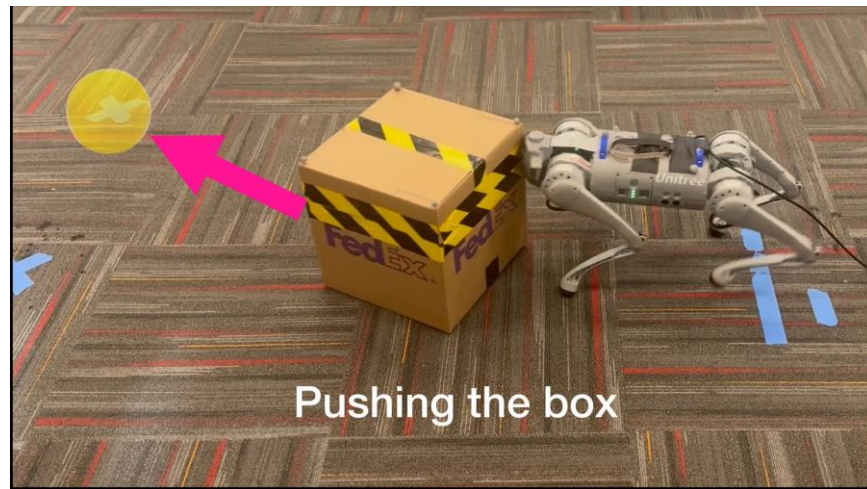
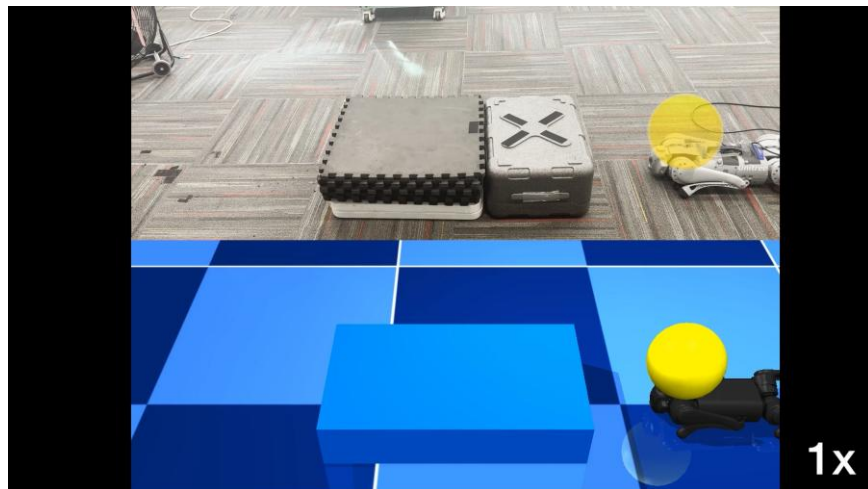
Runs on CPU, inputs are directly torque

MJPC

Bimanual Manipulation

with ALOHA

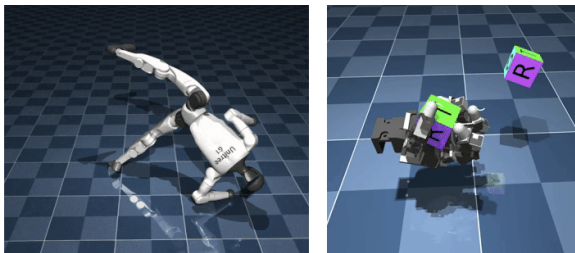
Runs on CPU, inputs are directly torque



State from real robot are overwritten in the simulator

Hydrax

Sampling-based model predictive control on GPU with [JAX](#) and [MuJoCo MJX](#).



About

Hydrax implements various sampling-based MPC algorithms on GPU. It is heavily inspired by [MJPC](#), but focuses exclusively on sampling-based algorithms, runs on hardware accelerators via JAX and MJX, and includes support for [online domain randomization](#).

Available methods:

Algorithm	Description	Import
Predictive sampling	Take the lowest-cost rollout at each iteration.	hydrax.algs.PredictiveSampling
MPPI	Take an exponentially weighted average of the rollouts.	hydrax.algs.MPPI
Cross Entropy Method	Fit a Gaussian distribution to the n best "elite" rollouts.	hydrax.algs.CEM
DIAL-MPC	MPPI with dual-loop, annealed sampling covariance.	hydrax.algs.DIAL
Evosax	Any of the 30+ evolution strategies implemented in evosax . Includes CMA-ES, differential evolution, and many more.	hydrax.algs.Evosax
MPPI-CMA	MPPI with an adaptive sampling distribution.	hydrax.algs.MppiCma
MTP	Mix structured tensor-sampled trajectories with a local CEM update for global exploration.	hydrax.algs.MTP
CBO	Simulate an SDE that pulls samples toward a consensus point.	hydrax.algs.MTP

[vincekurtz/hydrax: Sampling-based model predictive control on GPU with JAX/MJX](#)

- Sampling control is an easy alternative to gradient-based control
- It helps when gradient information is absent (e.g. contact!). **If you have a gradient, use it!**
- Limitations:
 - No constraints consideration
 - It works if a powerful enough hardware is there (strong CPU, GPU – no Nvidia Jetson)
- Future directions:
 - More GPU computing
 - **Collaboration between gradient and sampling is possible!**

- First sample u^*

$$\begin{aligned}\theta^* &= \bar{\theta} + \sum_{k=0}^K \omega^k \Delta \theta^k \\ u^* &= \pi(\theta^*, \hat{\mathbf{x}}, t_0)\end{aligned} \quad \longrightarrow \quad \text{MPPI}$$

- then differentiate over all the rollouts given the initial state

$$\begin{aligned}\mathbf{F} &= \frac{\partial u^*}{\partial \mathbf{x}_0} = \frac{\partial \pi}{\partial \theta} \frac{\partial \theta^*}{\partial \mathbf{x}_0} + \frac{\partial \pi}{\partial \mathbf{x}_0} \\ \mathbf{F} &= \sum_{k=0}^K \frac{\partial \pi}{\partial \theta} \Delta \theta^k \frac{\omega^k}{\lambda} \left(\frac{\partial J^k}{\partial \mathbf{x}_0} - \sum_{j=1}^K \omega^j \frac{\partial J^j}{\partial \mathbf{x}_0} \right) + \frac{\partial \pi}{\partial \mathbf{x}_0}\end{aligned} \quad \longrightarrow \quad \text{JAX Automatic Differentiation}$$

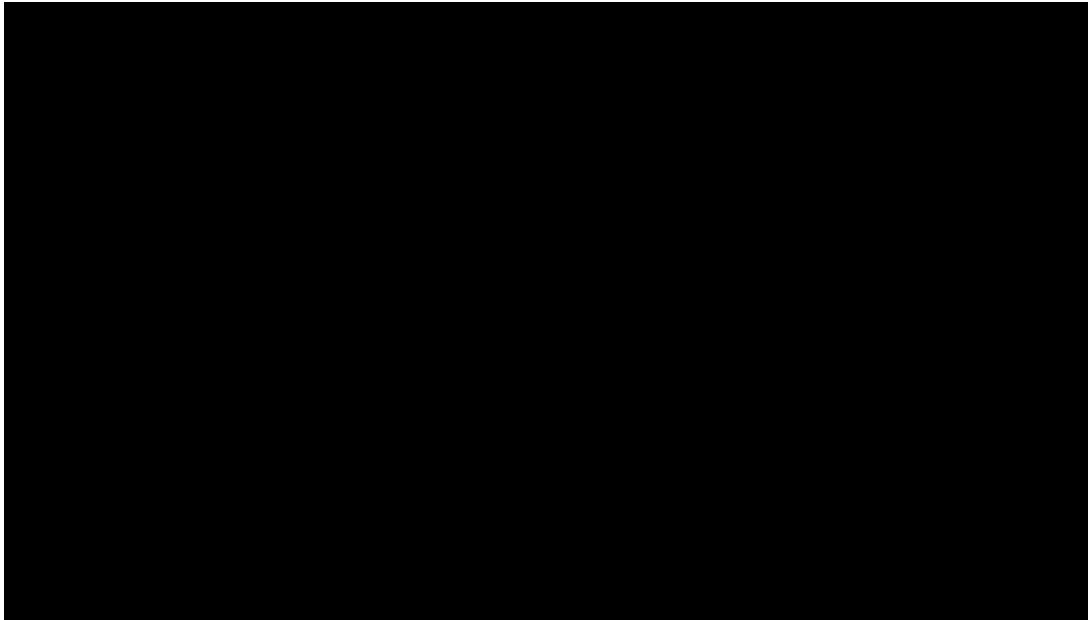
$$\mathbf{u} = \mathbf{u}^* + \mathbf{F} (\hat{\mathbf{x}} - \mathbf{x}_{\text{sp}})$$

- First sample u^* , then differentiate over all the rollouts

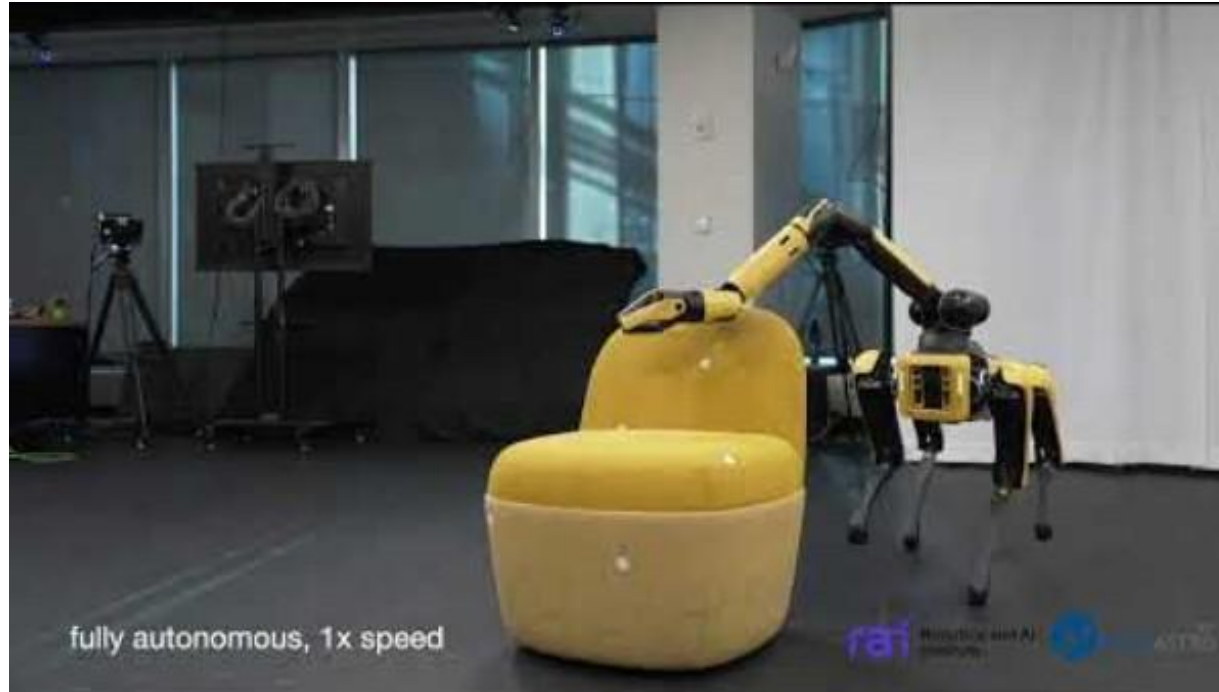


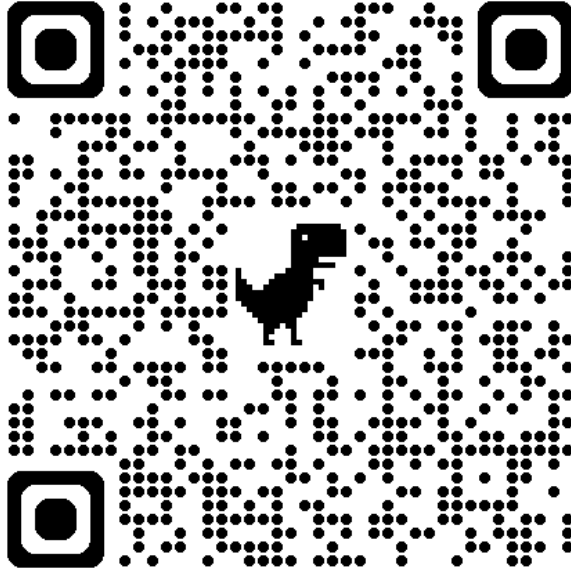
$$\mathbf{u} = \mathbf{u}^* + \mathbf{F}(\hat{\mathbf{x}} - \mathbf{x}_{sp})$$

- First sample u^* , then differentiate over all the rollouts



$$\mathbf{u} = \mathbf{u}^* + \mathbf{F}(\hat{\mathbf{x}} - \mathbf{x}_{sp})$$





<https://forms.gle/K1ZdD9s4t8UdVdD59>

Any question, or feedback **on this lesson** is welcome!
Completely anonymous!